



南京理工大学

2 数学基础与数据结构



2 数学基础与数据结构

- 集合、关系和函数
- 求和计算(*)
- 递归方程求解(*)
- 基本数据结构 (*)



函数

取整函数

- $\lfloor x \rfloor$: 不大于 x 的最大整数;
- $\lceil x \rceil$: 不小于 x 的最小整数。

例: 3.14, 3, -3.14, -3,

- $\lfloor 3.14 \rfloor = 3, \lceil 3.14 \rceil = 4, \lfloor -3.14 \rfloor = -4, \lceil -3.14 \rceil = -3.$
- 整数 x (无论正负) 的 $\lfloor x \rfloor$ 或 $\lceil x \rceil$ 值还是 x .



取整函数的若干性质

- $x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$;
- $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$;
- 对于 $n \geq 0$, $a, b > 0$, 有:
- $\lceil \lceil n/a \rceil / b \rceil = \lceil n/ab \rceil$;
- $\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$;
- $\lceil a/b \rceil \leq (a+(b-1))/b$; (函数值的紧上界)
- $\lfloor a/b \rfloor \geq (a-(b-1))/b$; (函数值的紧下界)
- $f(x) = \lfloor x \rfloor$, $g(x) = \lceil x \rceil$ 为单调递增函数



其他函数

- 指数函数

$$a > 1 \Rightarrow \lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0 \Rightarrow n^b = o(a^n)$$

- 对数函数vs.幂函数

$$\begin{aligned} \varepsilon > 0 \Rightarrow \lim_{n \rightarrow \infty} \frac{n^\varepsilon}{\log n} &= \lim_{n \rightarrow \infty} \frac{n^\varepsilon}{\ln n / \ln 2} = \ln 2 \times \lim_{n \rightarrow \infty} \frac{n^\varepsilon}{\ln n} \\ &= \ln 2 \times \lim_{n \rightarrow \infty} \frac{\varepsilon n^{\varepsilon-1}}{1/n} = \varepsilon \ln 2 \times \lim_{n \rightarrow \infty} n^\varepsilon = +\infty \Rightarrow \log n = o(n^\varepsilon) \end{aligned}$$



对数函数

$$\log_b a = \frac{\log_c a}{\log_c b} \quad (*)$$

$$a^{\log_b c} = c^{\log_b a} \quad (*)$$

$$\frac{\log_b a^{\log_b c}}{\log_b c^{\log_b a}} = \frac{\log_b c \square \log_b a}{\log_b a \square \log_b c} = 1$$



求和计算

- 问题：求 $\sum_{j=1}^n f(j)$
- 策略：用函数 $f(x)$ 积分来逼近
- 则若 $f(x)$ 递减，则

$$\int_m^{n+1} f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x)dx$$

- 若 $f(x)$ 递增，则

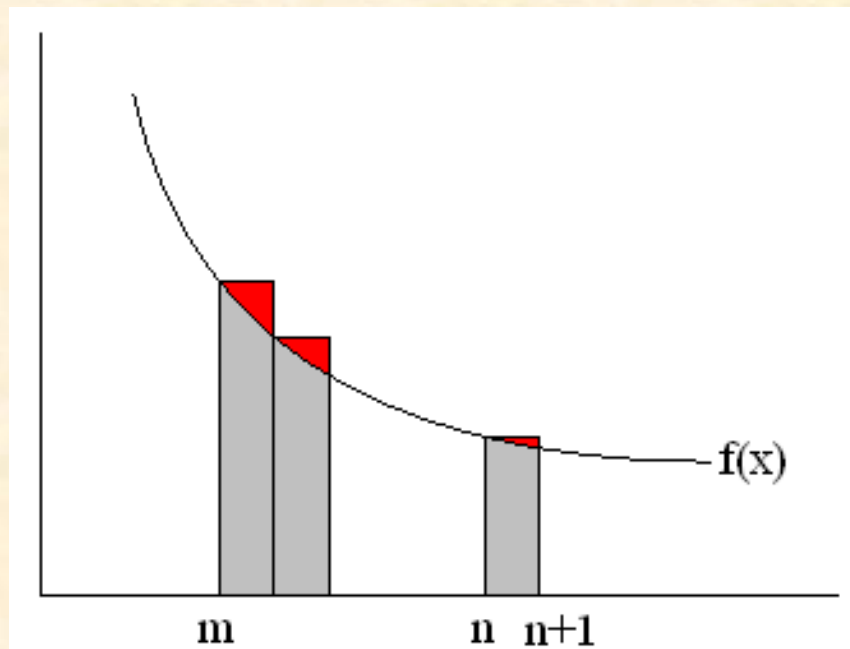
$$\int_{m-1}^n f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x)dx$$



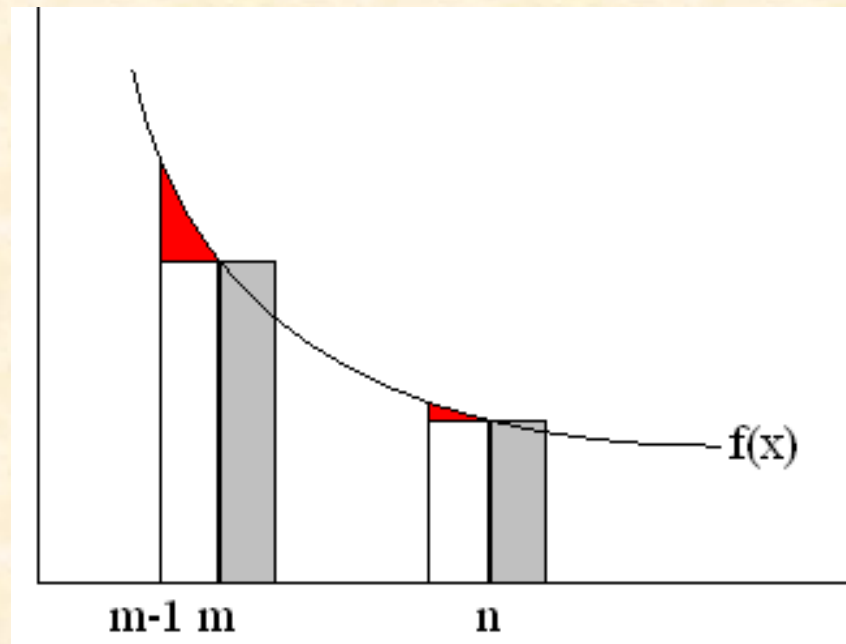
以 $f(x)$ 递减为例的图释

$$\sum_{j=m}^n f(j) = \sum_{x=m}^n f(x) \cdot \Delta x \quad (\Delta x = 1)$$

$$\int_m^{n+1} f(x) dx \leq \sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x) dx$$



$$\int_m^{n+1} f(x) dx \leq \sum_{j=m}^n f(j)$$

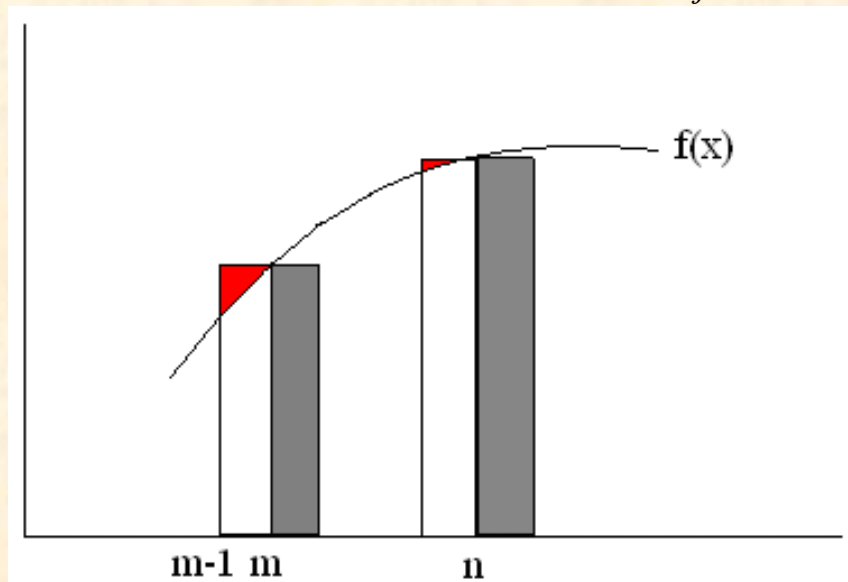


$$\sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x) dx$$

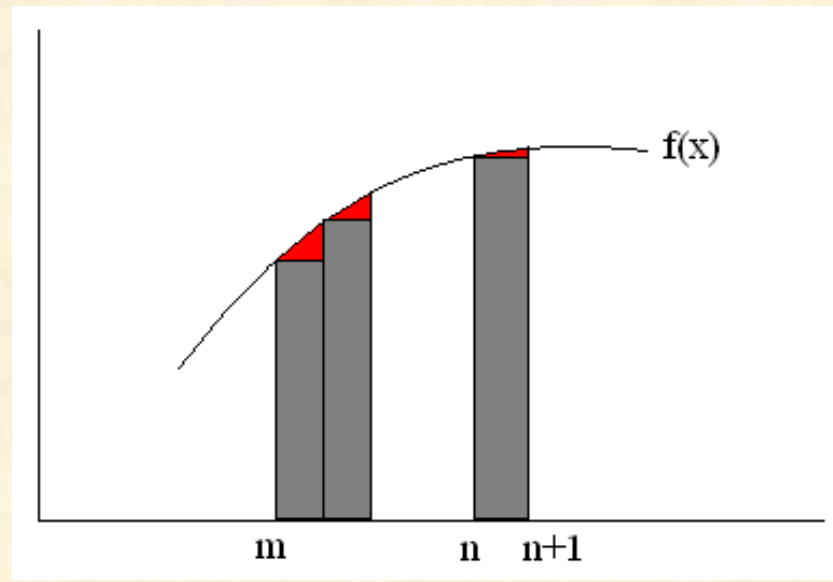


以 $f(x)$ 递增为例的图释

$$\int_{m-1}^n f(x) dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x) dx$$



$$\int_{m-1}^n f(x) dx \leq \sum_{j=m}^n f(j)$$



$$\sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x) dx$$



例: $\sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$

解: 因为

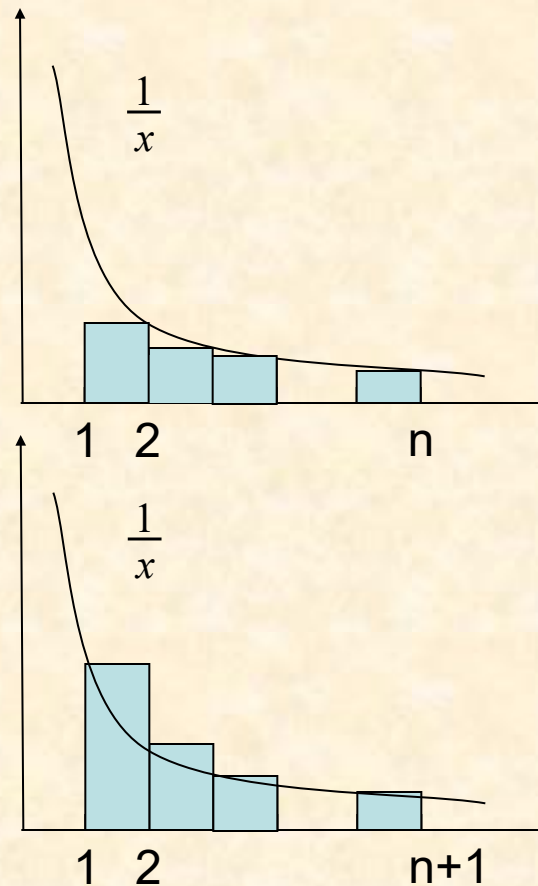
a. $\sum_{i=1}^n \frac{1}{i} = 1 + \sum_{i=2}^n \frac{1}{i} \leq 1 + \int_1^n \frac{1}{x} dx = 1 + \ln n$

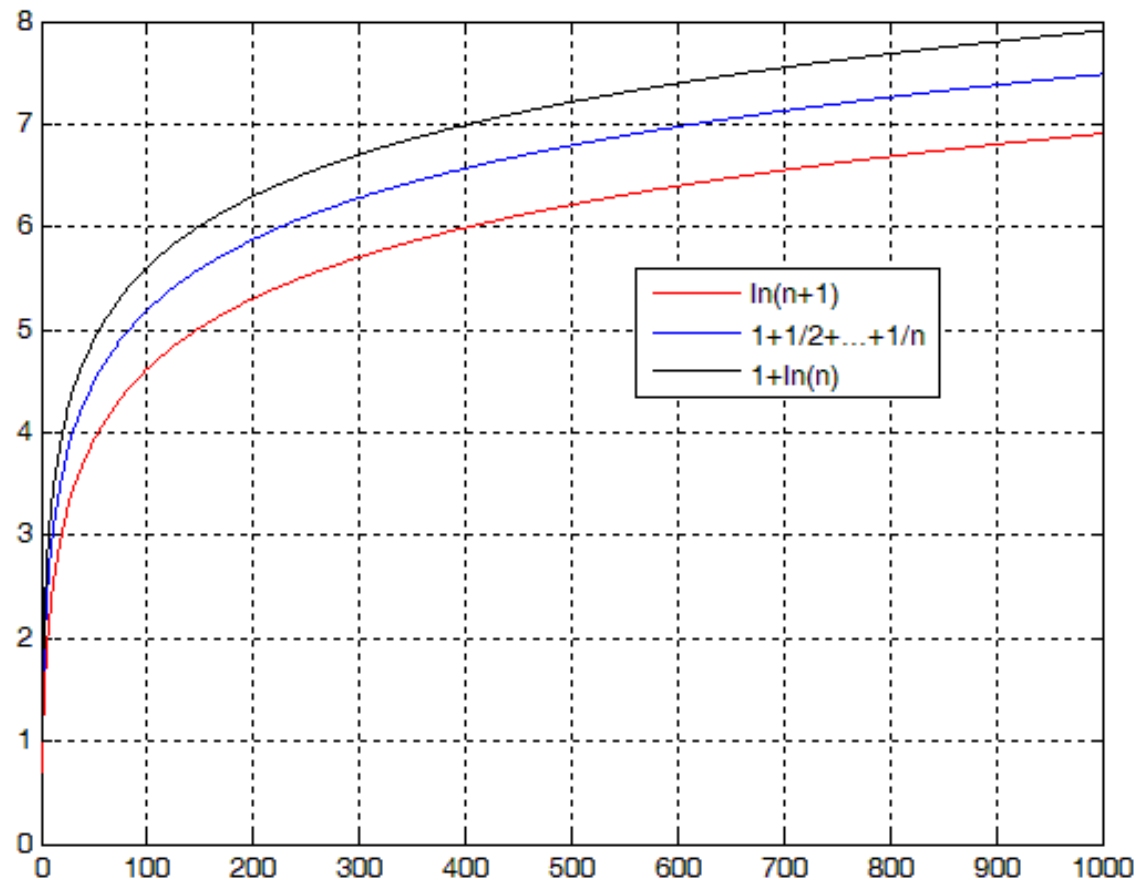
b. $\sum_{i=1}^n \frac{1}{i} \geq \int_1^{n+1} \frac{1}{x} dx = \ln(n+1)$

所以 $\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \ln n$

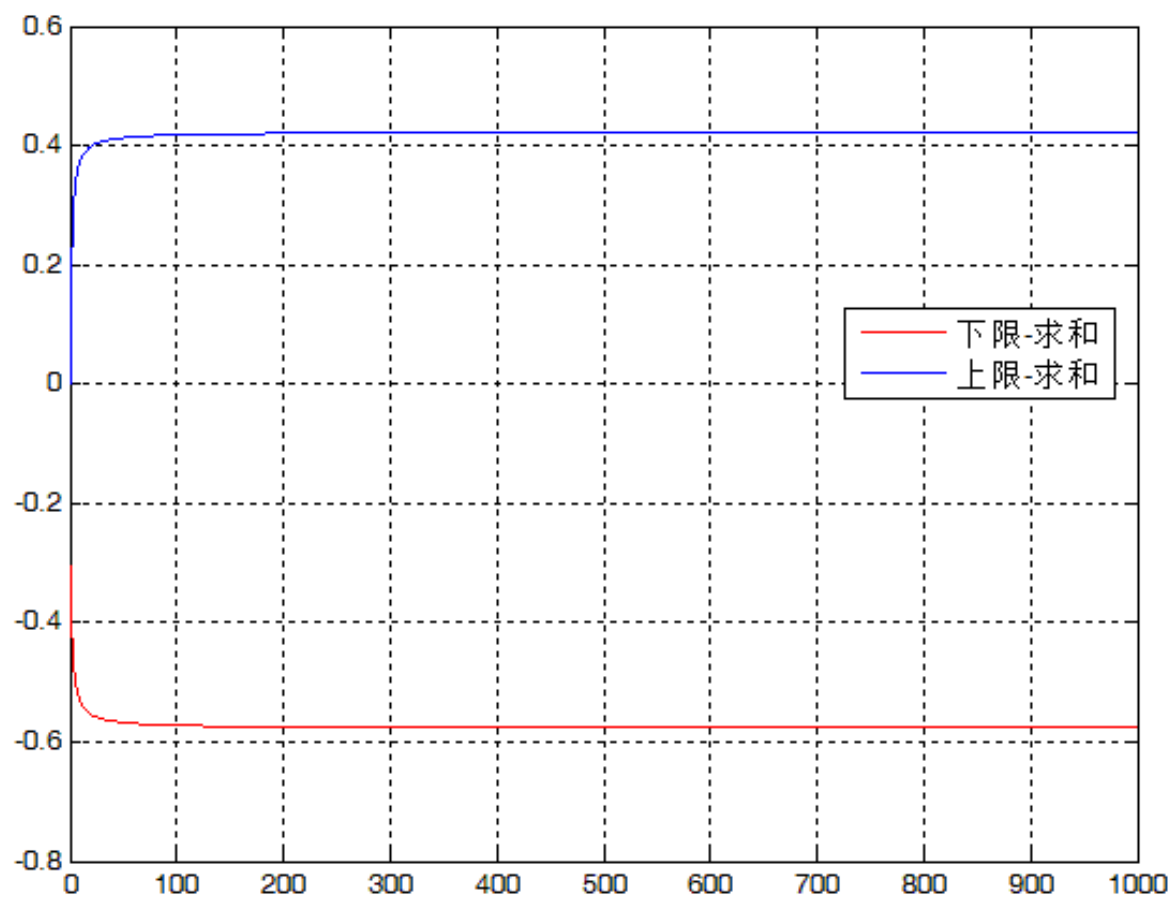
换底公式 $\frac{\log(n+1)}{\log e} \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \frac{\log n}{\log e}$

所以 $\sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$





$$\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \ln n$$



$$\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq 1 + \ln n$$



$$\text{例2} \quad \sum_{j=1}^n \log j = \Theta(n \log n) \quad \sum_{j=1}^n \log j = \log n + \sum_{j=1}^{n-1} \log j$$

$$\leq \log n + \int_1^n \log x dx$$

$$= \log n + \int_1^n \frac{\ln x}{\ln 2} dx = \log n + \frac{1}{\ln 2} \int_1^n \ln x dx$$

$$= \log n + \frac{1}{\ln 2} \left[x \ln x \Big|_1^n - \int_1^n x d \ln x \right]$$

$$= \log n + \frac{1}{\ln 2} \left[n \ln n - \int_1^n x \cdot \frac{1}{x} dx \right]$$

$$= \log n + \frac{1}{\ln 2} [n \ln n - (n-1)]$$

$$= \log n + n \log n - n \log e + \log e = O(n \log n)$$

$$\sum_{j=1}^n \log j = \sum_{j=2}^n \log j$$

$$\geq \int_1^n \log x dx$$

$$= n \log n - n \log e + \log e = \Omega(n \log n)$$

由 Θ 符号定义, 可知 $\sum_{j=1}^n \log j = \Theta(n \log n)$



递归方程求解

- 常系数线性同质递归方程 (linear homogeneous with constant coefficients)
- 几类特殊的非同质递归方程求解



常系数线性同质递归方程

$$T(n) = a_1T(n-1) + a_2T(n-2) + \cdots + a_kT(n-k)$$

称为k 阶常系数线性同质递归方程。

其特征方程(characteristic equation)为:

$$x^k = a_1x^{k-1} + a_2x^{k-2} + \cdots + a_k \quad \Rightarrow \quad x^k - a_1x^{k-1} - a_2x^{k-2} - \cdots - a_k = 0$$

我们仅仅关注一阶和二阶同质方程，因为对于使用此类递归方程来分析算法，往往是一阶或二阶的。对于一阶情形，求解过程非常直观(直接递推):

$$T(n) = aT(n-1) = a^2T(n-2) = \cdots = a^nT(0)$$

对于二阶情形，特征方程变为：

$$x^2 - a_1x - a_2 = 0$$

设该二次方程(quadratic equation)的两个根为： x_1, x_2 。那么 $T(n)$ 可以表示为：

$$T(n) = \begin{cases} c_1 \cdot x_1^n + c_2 \cdot x_2^n & , \text{if } x_1 \neq x_2 \\ c_1 \cdot r^n + c_2 \cdot n \cdot r^n & , \text{if } x_1 = x_2 = r \end{cases}$$

然后利用初始值： $T(n_0)$ 及 $T(n_0 + 1)$ ，使用待定系数法解出系数即可。



例：

已知 $T(n) = 3T(n-1) + 4T(n-2)$ 并且 $T(0) = 1, T(1) = 4$

解：特征方程为 $x^2 - 3x - 4 = 0 \Rightarrow x_1 = -1, x_2 = 4$

$$\therefore T(n) = c_1(-1)^n + c_2 4^n \Rightarrow \begin{cases} T(0) = 1 = c_1 + c_2 \\ T(1) = 4 = -c_1 + 4c_2 \end{cases}$$

$$\Rightarrow c_1 = 0, c_2 = 1 \Rightarrow T(n) = 4^n$$



例：

已知 $T(n) = T(n-1) + T(n-2)$ 并且 $T(1) = T(2) = 1$

解：特征方程为 $x^2 - x - 1 = 0 \Rightarrow x_1 = \frac{1+\sqrt{5}}{2}, x_2 = \frac{1-\sqrt{5}}{2}$

$$\therefore T(n) = c_1 \left(\frac{1+\sqrt{5}}{2} \right)^n + c_2 \left(\frac{1-\sqrt{5}}{2} \right)^n$$

$$\Rightarrow c_1 = \frac{1}{\sqrt{5}}, c_2 = -\frac{1}{\sqrt{5}}$$

$$\Rightarrow T(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n$$



非同质递归方程的求解

$$T(n) = T(n-1) + g(n), n \geq 1 \quad \Rightarrow \quad T(n) = T(0) + \sum_{i=1}^n g(i)$$

$$T(n) = g(n)T(n-1), n \geq 1 \quad \Rightarrow \quad T(n) = g(n)g(n-1) \cdots g(1)T(0)$$

$$T(n) = g(n)T(n-1) + h(n), n \geq 1 \quad \Rightarrow \quad ?$$



$$T(n) = g(n)T(n-1) + h(n), n \geq 1 \quad \text{且 } T(0) \text{ 已知。}$$

$$\text{解: 令 } k(n) = \begin{cases} \frac{T(n)}{g(n)g(n-1)\cdots g(1)}, n \geq 1 \\ T(0), n = 0 \end{cases}$$

$$k(n) = \frac{T(n)}{g(n)g(n-1)\cdots g(1)}, n \geq 1 \quad \Rightarrow \quad T(n) = g(n)g(n-1)\cdots g(1)k(n)$$

$$\Rightarrow T(n-1) = g(n-1)g(n-2)\cdots g(1)k(n-1)$$

$$\Rightarrow \underbrace{g(n)g(n-1)\cdots g(1)k(n)}_{T(n)} = g(n) \underbrace{g(n-1)\cdots g(1)k(n-1)}_{T(n-1)} + h(n)$$

$$\Rightarrow k(n) = k(n-1) + \frac{h(n)}{g(n)\cdots g(1)}$$



$$k(n) = k(n-1) + \frac{h(n)}{g(n) \cdots g(1)}$$

$$\Rightarrow k(n) = k(n-2) + \frac{h(n-1)}{g(n-1) \cdots g(1)} + \frac{h(n)}{g(n) \cdots g(1)}$$

$$\Rightarrow k(n) = k(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1) \cdots g(1)} = T(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1) \cdots g(1)}$$

$$\Rightarrow T(n) = g(n)g(n-1) \cdots g(1) \left(T(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1) \cdots g(1)} \right)$$



例: $T(n) = nT(n-1) + n!, T(0) = 0$

解: 这里 $g(n) = n, h(n) = n!$

$$\begin{aligned} T(n) &= g(n)g(n-1)\cdots g(1)\left(T(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)}\right) \\ &= n(n-1)\cdots 1\left(0 + \sum_{i=1}^n \frac{i!}{i(i-1)\cdots 1}\right) = n!n \end{aligned}$$



展开递推式

引理 1.1 下面递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ af(n/c) + bn^x & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = bn^x \log_c n + dn^x \quad \text{if } a = c^x$$

$$f(n) = (d + \frac{bc^x}{a - c^x})n^{\log_c a} - (\frac{bc^x}{a - c^x})n^x \quad \text{if } a \neq c^x$$

其中, a, c 是非负整数, b, d, x 是非负常数, 且对于某个非负整数 k 有 $n=c^k$.

证明: $f(n)=af(n/c)+bn^x$

$$=a\{af(n/c^2)+b(n/c)^x\}+bn^x$$

$$=a^2f(n/c^2) + (a/c^x)bn^x + bn^x$$

$$=a^2\{af(n/c^3)+b(n/c^2)^x\} + (a/c^x)bn^x + bn^x$$

$$=a^3f(n/c^3) + (a/c^x)^2bn^x + (a/c^x)bn^x + bn^x$$



= ...

$$= a^k f(n/c^k) + (a/c^x)^{k-1} bn^x + \dots + (a/c^x) bn^x + bn^x$$

$$= a^k d + bn^x \sum_{j=0}^{k-1} (a/c^x)^j \quad (j=0, \dots, k-1)$$

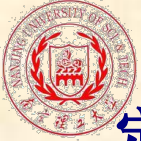
$$= d n^{\log_c a} + bn^x \sum_{j=0}^{k-1} (a/c^x)^j \quad (\because a^{\log_c n} = n^{\log_c a})$$

若 $a=c^x$ ，则 $\sum_{j=0}^{k-1} (a/c^x)^j = k = \log_c n$ ， $f(n) = dn^x + bn^x \cdot \log_c n$

若 $a \neq c^x$ ，则可整理为 $f(n) = (d + \frac{bc^x}{a - c^x}) n^{\log_c a} - (\frac{bc^x}{a - c^x}) n^x$

复杂度函数的阶取决于指数 x 和 $\log_c a$ 的大小(c, a 的选择)

将本例中的 x 和 $\log_c a$ 的关系作一深入讨论，有如下定理：



定理 1.3 设 a, c 是非负整数, b, d, x 是非负常数, 且对于某个非负整数 k 有 $n=c^k$. 那么, 对于递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ af(n/c) + bn^x & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = \begin{cases} \Theta(n^x) & \text{if } a < c^x \\ \Theta(n^x \log n) & \text{if } a = c^x \\ \Theta(n^{\log_c a}) & \text{if } a > c^x \end{cases}$$

特别地, 若 $x=1$, 则

$$f(n) = \begin{cases} \Theta(n) & \text{if } a < c \\ \Theta(n \log n) & \text{if } a = c \\ \Theta(n^{\log_c a}) & \text{if } a > c \end{cases}$$



Master Theorem

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

$$T(n) = aT(n/b) + f(n)$$

$$= a\{aT(n/b^2) + f(n/b)\} + f(n)$$

$$= a^2T(n/b^2) + af(n/b) + f(n)$$

$$= a^2\{aT(n/b^3) + f(n/b^2)\} + af(n/b) + f(n)$$

$$= a^3T(n/b^3) + a^2f(n/b^2) + af(n/b) + f(n)$$

$$= \dots$$

$$= a^kT(n/b^k) + \sum_{j=0}^{k-1} a^j f(n/b^j) \quad \text{// 设 } b^k = n, \text{ 则 } k = \log_b n$$

$$= a^{\log_b n} + \sum_{i=0}^{\log_b n - 1} a^i f(n/b^i)$$

$$= n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f(n/b^i)$$



Master Theorem

设 $a \geq 1, b > 1$ 为常数, $f(n)$ 为一给定的函数, $T(n)$ 递归定义如下:

$$T(n) = a \cdot T(n/b) + f(n)$$

并且 $T(n)$ 有适当的初始值。那么, 当 n 充分大时, 有:

1) 若有 $\varepsilon > 0$, 使 $f(n) = O(n^{(\log_b a) - \varepsilon})$

(即 $f(n)$ 的量级多项式地小于 $n^{\log_b a}$ 的量级), 则 $T(n) = \Theta(n^{\log_b a})$ 。

2) 若 $f(n) = \Theta(n^{\log_b a})$

(即 $f(n)$ 的量级等于 $n^{\log_b a}$ 的量级), 则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

3) 若有 $\varepsilon > 0$, 使 $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$

(即 $f(n)$ 的量级多项式地大于 $n^{\log_b a}$ 的量级), 且满足正规性条件:

存在常数 $c < 1$, 使得对所有足够大的 n , 有 $a \cdot f(n/b) \leq c \cdot f(n)$,

则 $T(n) = \Theta(f(n))$ 。

正规性条件的直观含义:

对所有足够大的 n , a 个子问题的分解准备与再组合所需要的时间总和, 严格小于原问题的分解准备和组合所需要的时间。



Master Theorem

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

$$T(n) = aT(n/b) + f(n)$$

$$= a\{aT(n/b^2) + f(n/b)\} + f(n)$$

$$= a^2T(n/b^2) + af(n/b) + f(n) \quad // \quad a * f(n/b) \leq c * f(n)$$

$$= a^2\{aT(n/b^3) + f(n/b^2)\} + af(n/b) + f(n)$$

$$= a^3T(n/b^3) + a^2f(n/b^2) + af(n/b) + f(n)$$

$$= \dots // a^2f(n/b^2) = a\{af(n/b^2)\} \leq a\{c * f(n/b)\} = c\{a * f(n/b)\} \leq c^2 * f(n) < c * f(n)$$

$$= a^kT(n/b^k) + \sum a^j f(n/b^j) \quad \{j=0, \dots, k-1\} \quad // \text{设 } b^k = n, \text{ 则 } k = \log_b n$$

$$\leq a^{\log_b n} + f(n) * \sum c^j \quad \{j=0, \dots, k-1\}$$

$$< n^{\log_b a} + f(n) * (1/(1-c))$$



几个例子

$$T(n) = 9T(n/3) + n$$

$$\because a = 9, b = 3, f(n) = n \quad \therefore \log_b a = 2 \quad \therefore \exists \varepsilon = 1$$

$$f(n) = O(n^{\log_b a - \varepsilon}) \quad \therefore T(n) = \Theta(n^2)$$

$$T(n) = T(2n/3) + 1$$

$$\because a = 1, b = \frac{3}{2}, f(n) = 1 \quad \therefore \log_b a = 0$$

$$f(n) = \Theta(n^{\log_b a}) \quad \therefore T(n) = \Theta(\log n)$$

$$T(n) = 3T(n/4) + n \log n$$

$$\because a = 3, b = 4, \log_b a \approx 0.793, f(n) = n \log n$$

$$\therefore \exists \varepsilon = 0.21, f(n) = n \log n = \Omega(n^{0.79+0.21})$$

$$\therefore T(n) = \Theta(n \log n)$$



代入法求解递推式

例 1.36 考虑下面递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

其中 b, d 是非负常数。当 n 是 2 的幂时，递推式可简化为

$$f(n) = 2f(n/2) + bn$$

由引理 1.1，其解为 $bn \log n + dn$ 。我们猜测，对于某常数 $c > 0$ ， $f(n) \leq cbn \log n + dn$ ，再确定 c 的值。假设这个断言对于 $n \geq 2$ 均成立。

$$\begin{aligned} f(n) &= f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn \\ &\leq cb \lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + d \lfloor n/2 \rfloor + cb \lceil n/2 \rceil \log \lceil n/2 \rceil + d \lceil n/2 \rceil + bn \\ &\leq cb \lfloor n/2 \rfloor \log \lceil n/2 \rceil + cb \lceil n/2 \rceil \log \lceil n/2 \rceil + dn + bn \\ &= cbn \log \lceil n/2 \rceil + dn + bn \\ &\leq cbn \log \frac{n+1}{2} + dn + bn \\ &= cbn \log(n+1) - cbn + dn + bn \end{aligned}$$



$$f(n) \leq cbn \log(n+1) - cbn + dn + bn$$

为了证明 $f(n) \leq cbn \log n + dn$

必有

$$cbn \log(n+1) - cbn + dn + bn \leq cbn \log n + dn$$

即

$$c \geq \frac{1}{1 + \log \frac{n}{n+1}} = \frac{1}{1 + \log \frac{1}{1 + \frac{1}{n}}} \leq \frac{1}{1 + \log \frac{1}{1 + \frac{1}{2}}} \approx 2.4094$$

取 $c=2.41$ ，有

$$f(n) \leq 2.41bn \log n + dn$$



代入法求解递推式

例 1.37 考虑下面递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

其中 b, d 是非负常数。我们猜测，对于某常数 $c > 0$ ，
 $f(n) \geq cbn \log n + dn$ ，再确定 c 的值。

详见pp.38.

$$f(n) \geq \frac{bn \log n}{2} + dn$$

对所有的 $n \geq 1$ 均成立.



定理 1.4 设

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

其中 b, d 是非负常数。那么有

$$f(n) = \Theta(n \log n)$$



例 1.39 考虑下面的递推式

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ b & \text{if } n = 1 \\ f(\lfloor c_1 n \rfloor) + f(\lfloor c_2 n \rfloor) + bn & \text{if } n \geq 2 \end{cases}$$

其中 b, c_1, c_2 是正常数, 且 $c_1 + c_2 < 1$.

解: 作为特例, 当 n 是 2 的幂, 且 $c_1 = c_2 = 1/4$, 问题化为

$$f(n) = \begin{cases} b & \text{if } n = 1 \\ 2f(n/4) + bn & \text{if } n \geq 2 \end{cases}$$

由引理 1.1, 其解为 $f(n) = 2bn - b\sqrt{n}$.

猜测对于某个 $c > 0$, 当 $c_1 + c_2 < 1$ 时, $f(n)$ 有上界即 $f(n) \leq cbn$,
(c 值待定)。



$$\begin{aligned}f(n) &= f(\lfloor c_1 n \rfloor) + f(\lfloor c_2 n \rfloor) + bn \\&\leq cb \lfloor c_1 n \rfloor + cb \lfloor c_2 n \rfloor + bn \\&\leq cbc_1 n + cbc_2 n + bn \\&= c(c_1 + c_2)bn + bn\end{aligned}$$

为了证明 $f(n) \leq cbn$, 必须有

$$c(c_1 + c_2)bn + bn \leq cbn$$

即

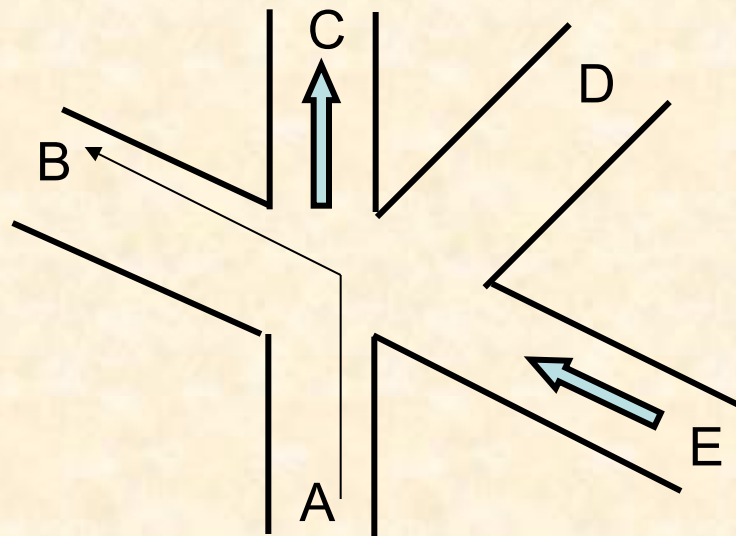
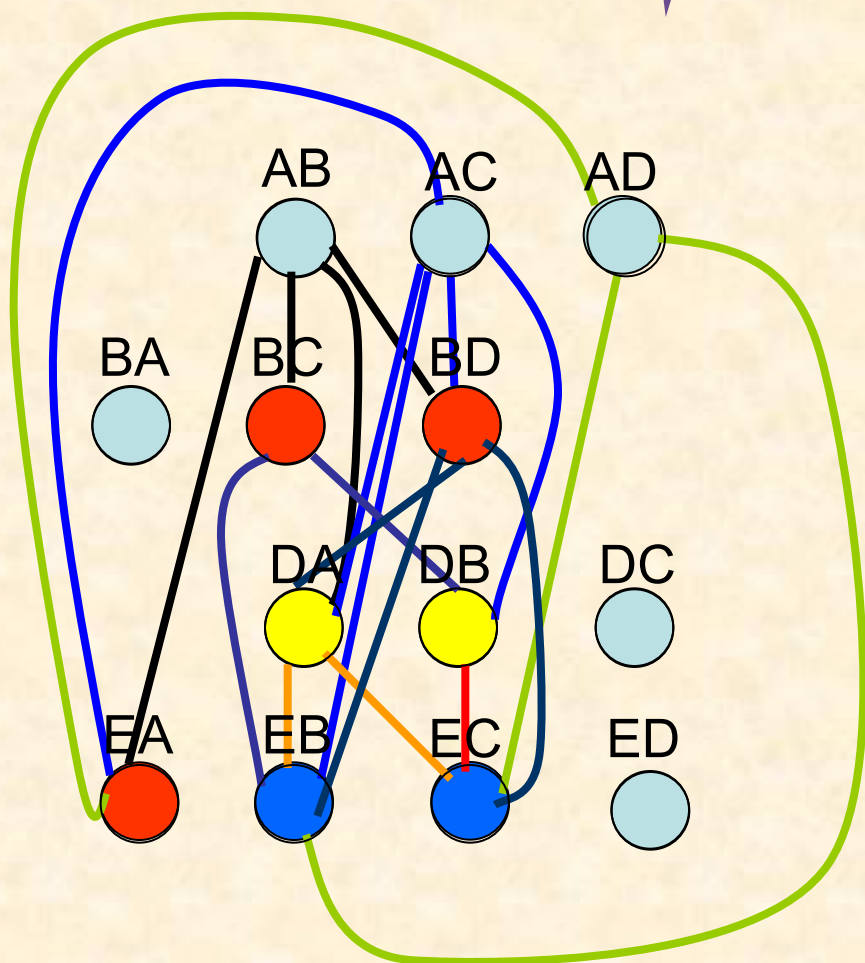
$$c \geq \frac{1}{1 - c_1 - c_2}$$

于是

$$f(n) \leq \frac{bn}{1 - c_1 - c_2} \quad \text{易验证, 对所有 } n \geq 0 \text{ 均成立。}$$

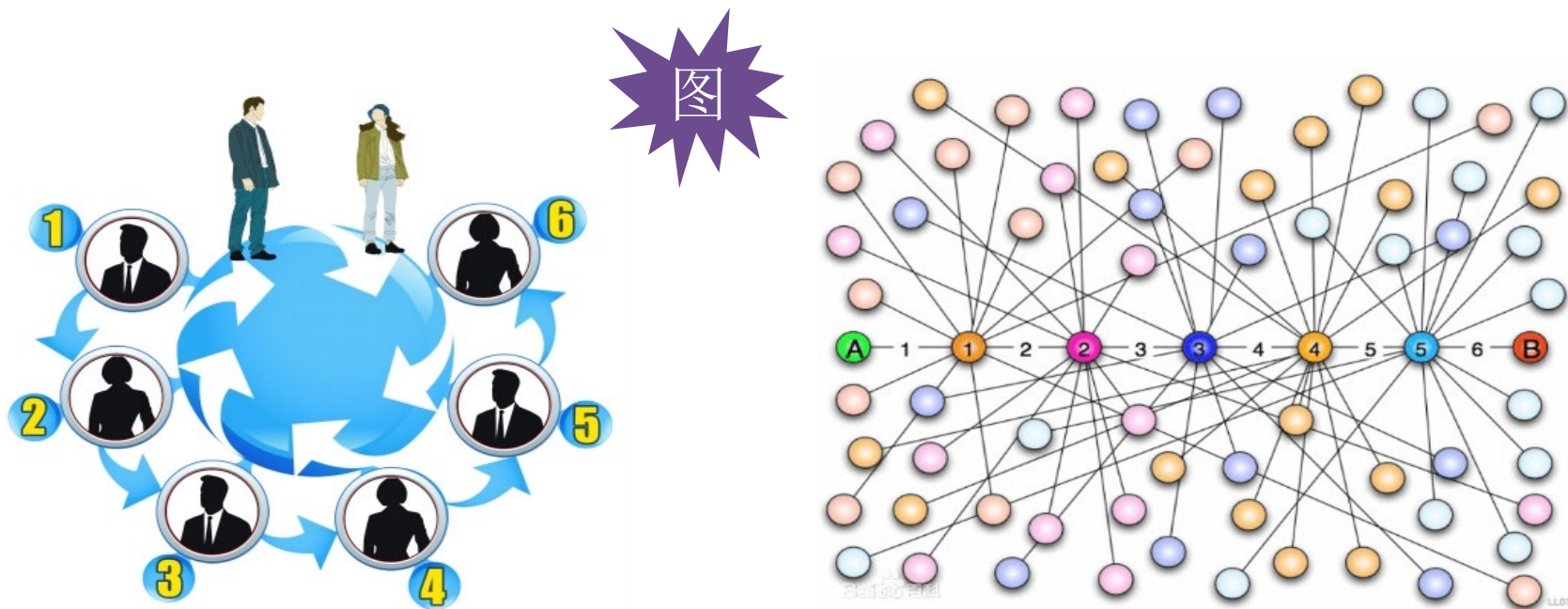


数据结构引例：多叉路口交通灯管理问题



顶点：一条通路
连线：不能同时通行
染色：有连线的两个顶点
不能具有相同颜色

▶▶▶ 数据结构引例：六度空间理论



你和任何一个陌生人之间所间隔的人不会超过6个，也就是说，最多通过6个中间人你就能够认识任何一个陌生人。



数据结构

- 算法的实现离不开数据结构。选择一个合适的数据结构对设计一个有效的算法有十分重要的影响。结构化程序设计创始人Niklaus Wirth(瑞士苏黎士高工)提出一个著名的论断：“程序=算法+数据结构”。1984年，Wirth因开发了Euler、Pascal等一系列崭新的计算语言而荣获图灵奖,有“结构化程序设计之父”之美誉。
- 本章我们将回顾几种重要的数据结构，包括二叉树、堆、不相交集。

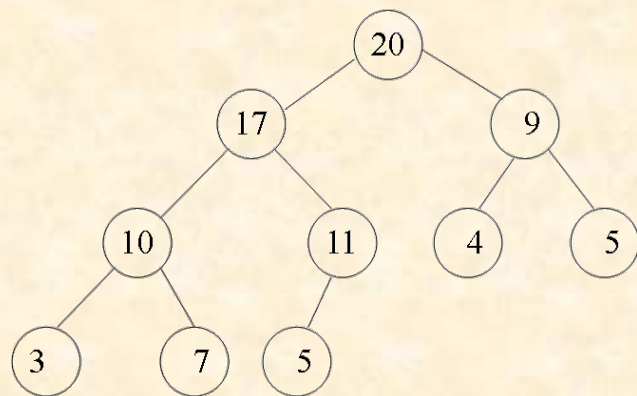


堆(Heap)

- 在许多算法中，需要大量用到如下两种操作：插入元素和寻找最大(小)值元素。为了提高这两种运算的效率，必须使用恰当的数据结构。
 - 普通队列：易插入元素，但求最大值元素需要搜索整个队列。
 - 排序数组：易找到最大值，但插入元素需要移动大量元素。
 - 堆则是一种有效实现上述两种运算的简单数据结构。



- 堆的定义：堆是一个几乎完全的二叉树，每个节点都满足这样的特性：任一父节点的键值(**key**)不小于子节点的键值。

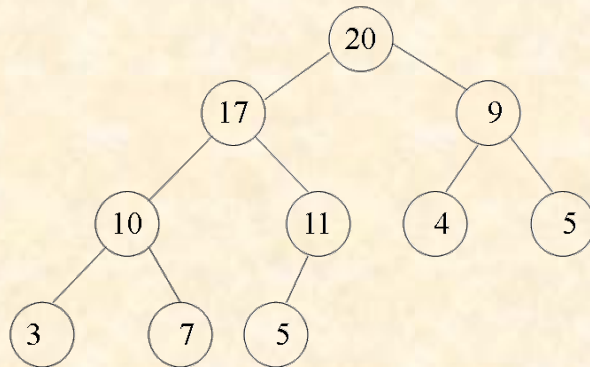


- 有 n 个节点的堆 T ,可以用一个数组 $H[1...n]$ 用下面的方式来表示：
 - T 的根节点存储在 $H[1]$ 中
 - 假设 T 的节点 x 存储在 $H[j]$ 中，那么，它的左右子节点分别存放在 $H[2j]$ 及 $H[2j+1]$ 中(如果有的话)。
 - $H[j]$ 的父节点如果不是根节点，则存储在 $H[\lfloor j/2 \rfloor]$ 中。

20	17	9	10	11	4	5	3	7	5
1	2	3	4	5	6	7	8	9	10



20	17	9	10	11	4	5	3	7	5
1	2	3	4	5	6	7	8	9	10



观察结论：

- 根节点键值最大，叶子节点键值较小。从根到叶子，键值以非升序排列。
- 节点的左右儿子节点键值并无顺序要求。
- 堆的数组表示呈“基本有序”状态。相应地，并非节点的高度越高，键值就越大。

**堆：名义上（逻辑上）是一棵树，
实际上（物理上）是个数组。**



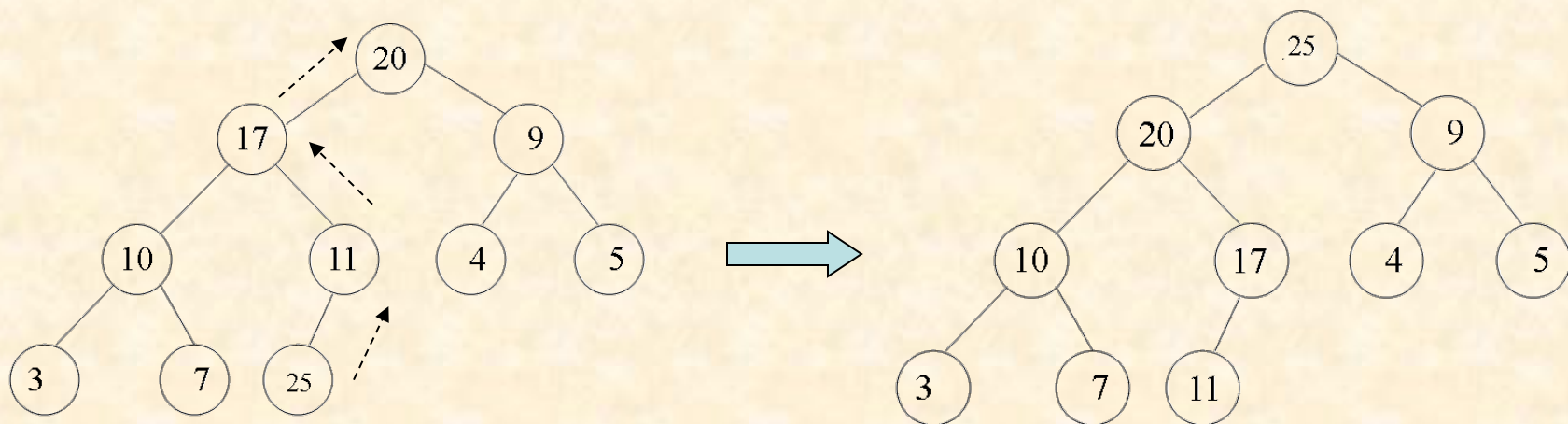
堆的基本操作

- `make-heap(A)`: 从数组A创建堆
- `insert(H,x)`: 插入元素x到堆H中
- `delete(H,i)`: 删除堆H的第i项
- `delete-max(H)`: 从非空堆H中删除最大键值并返回数据项



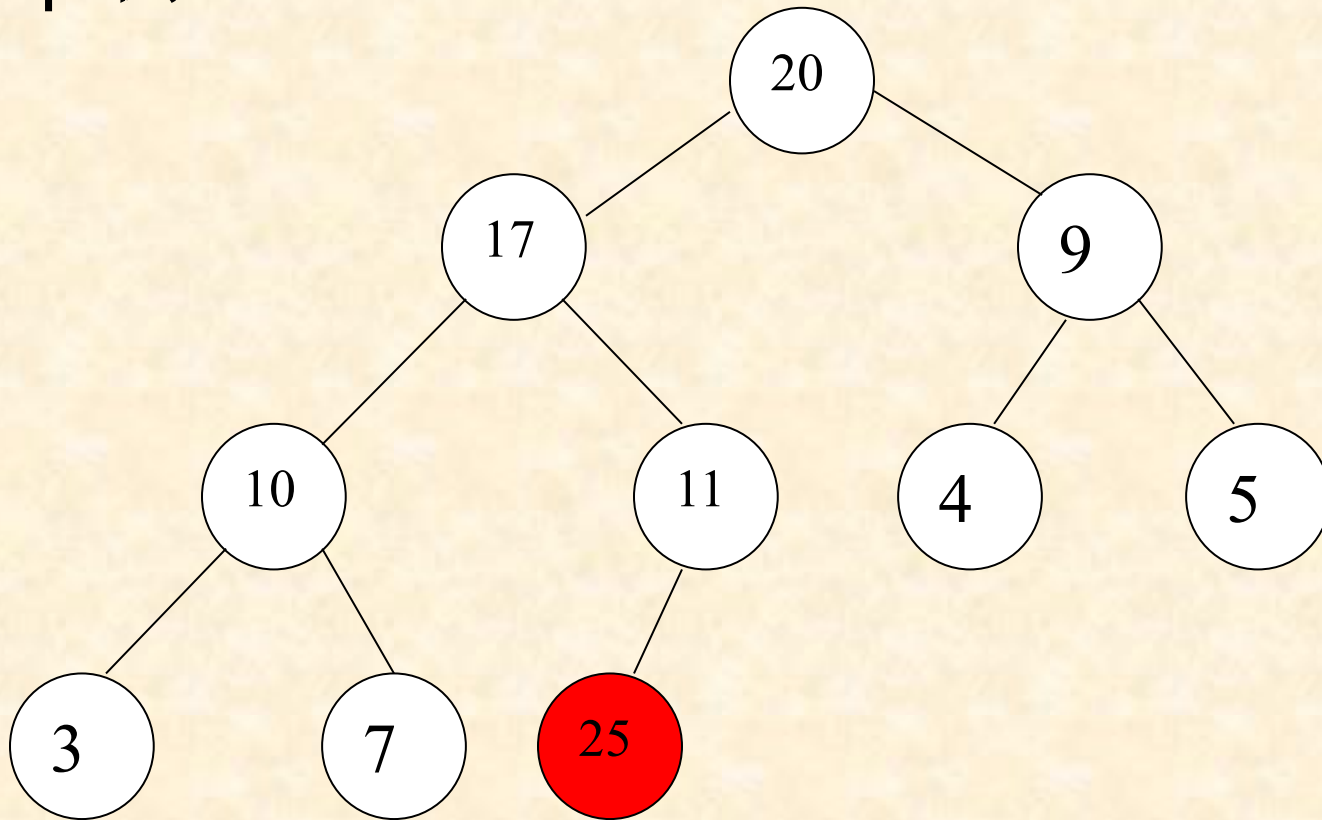
辅助运算Sift-up

- 若某个节点 $H[i]$ 键值大于其父节点的键值，就违背了堆的特性，需要进行调整。
- 调整方法：上移。
- 沿着 $H[i]$ 到根节点的唯一一条路径，将 $H[i]$ 移动到合适的位置上：比较 $H[i]$ 及其父节点 $H[\lfloor i/2 \rfloor]$ 的键值，若 $\text{key}(H[i]) > \text{key}(H[\lfloor i/2 \rfloor])$ ，则二者进行交换，直到 $H[i]$ 到达合适位置。



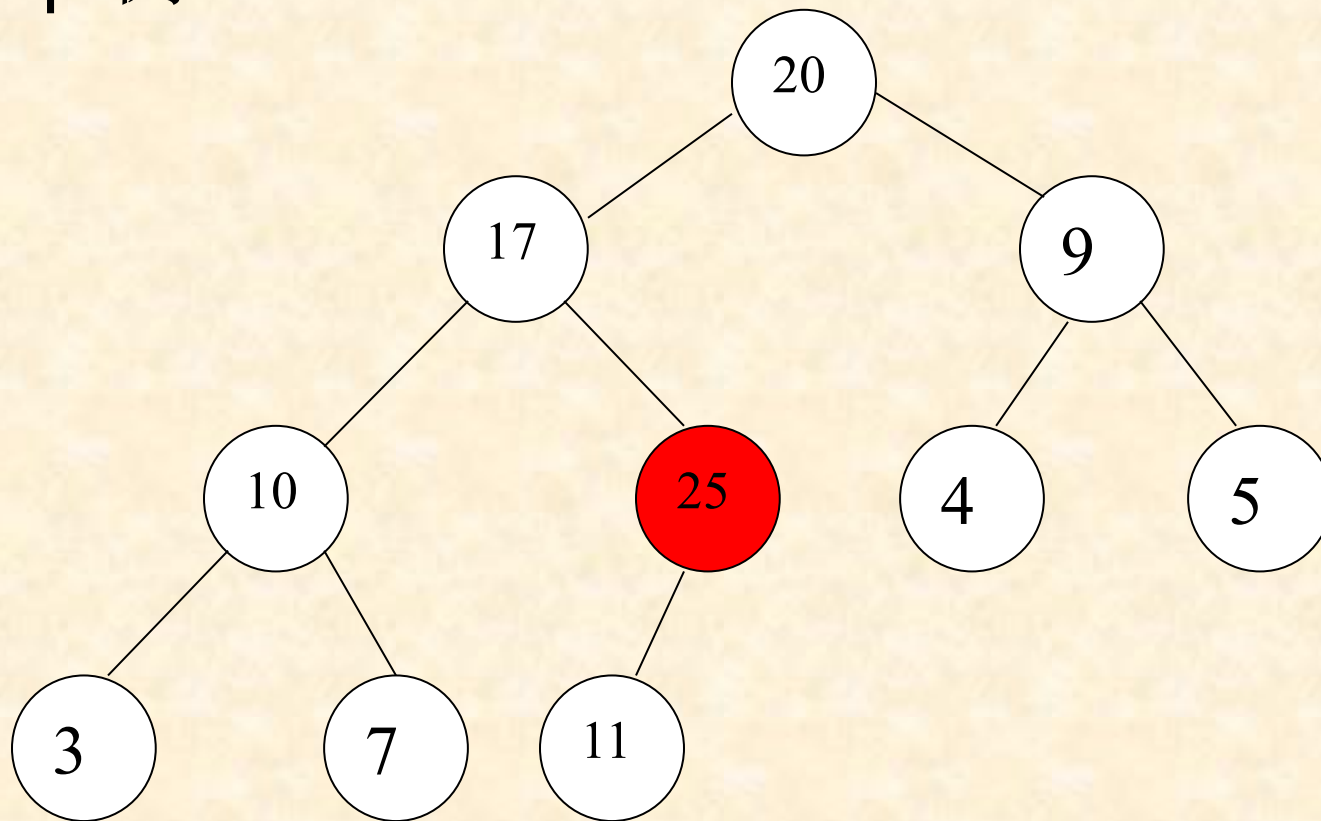


Sift-up 例1



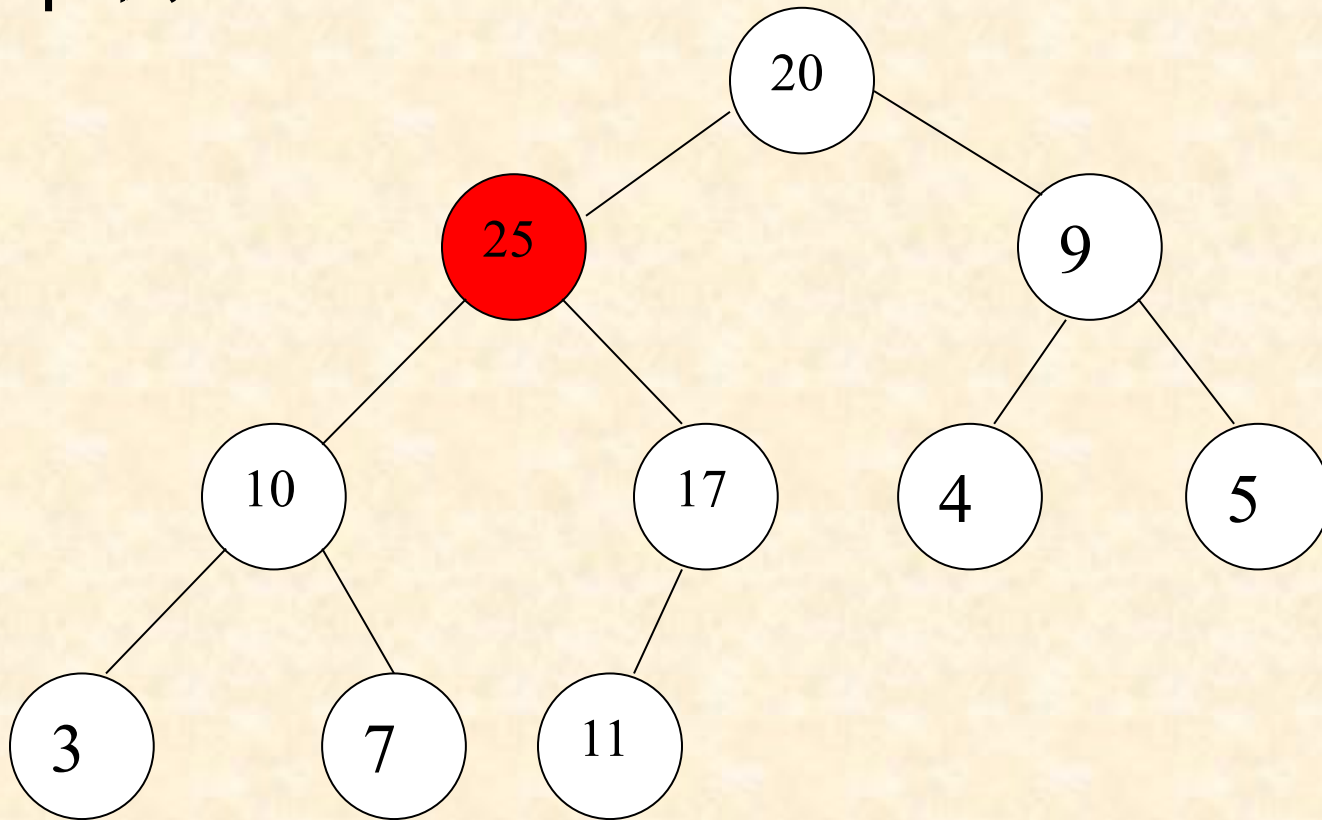


Sift-up 例1



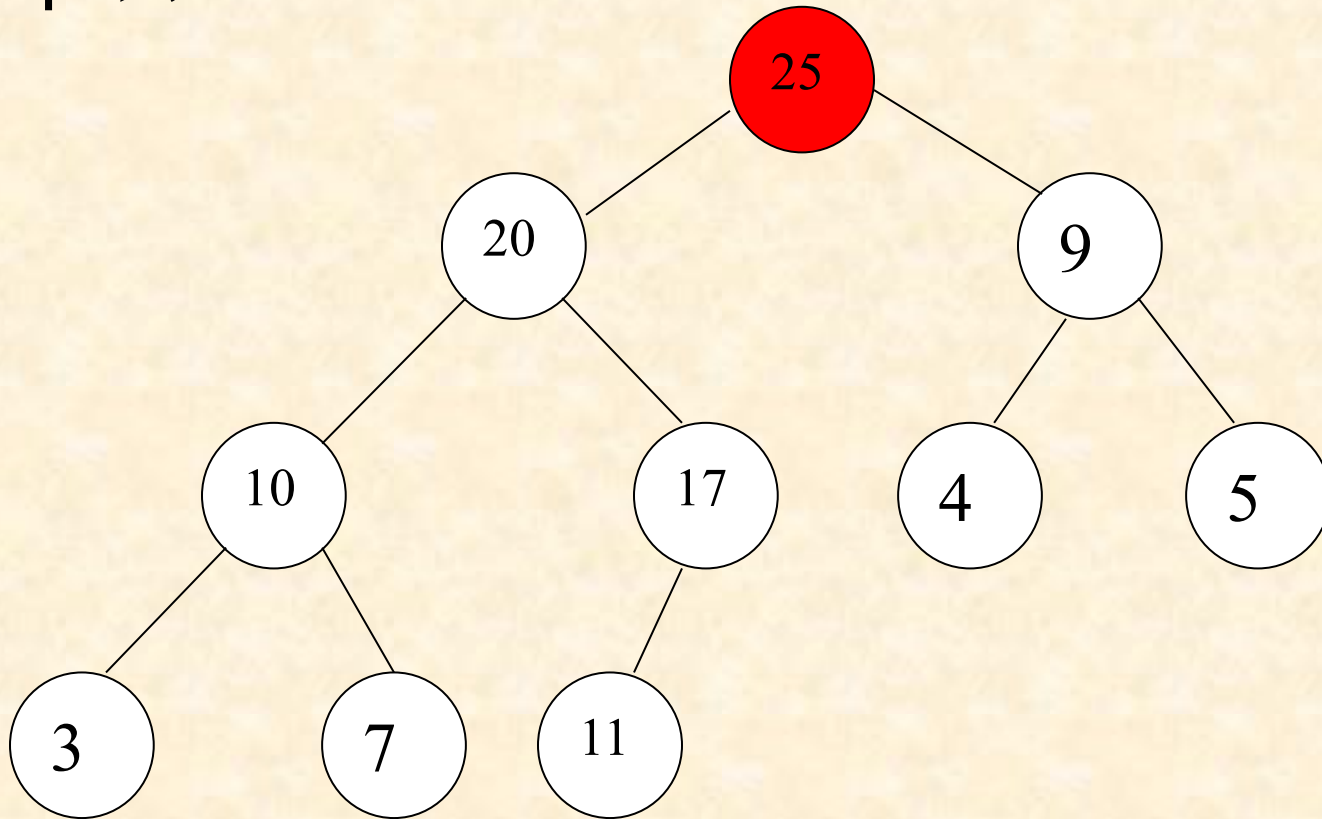


Sift-up 例1



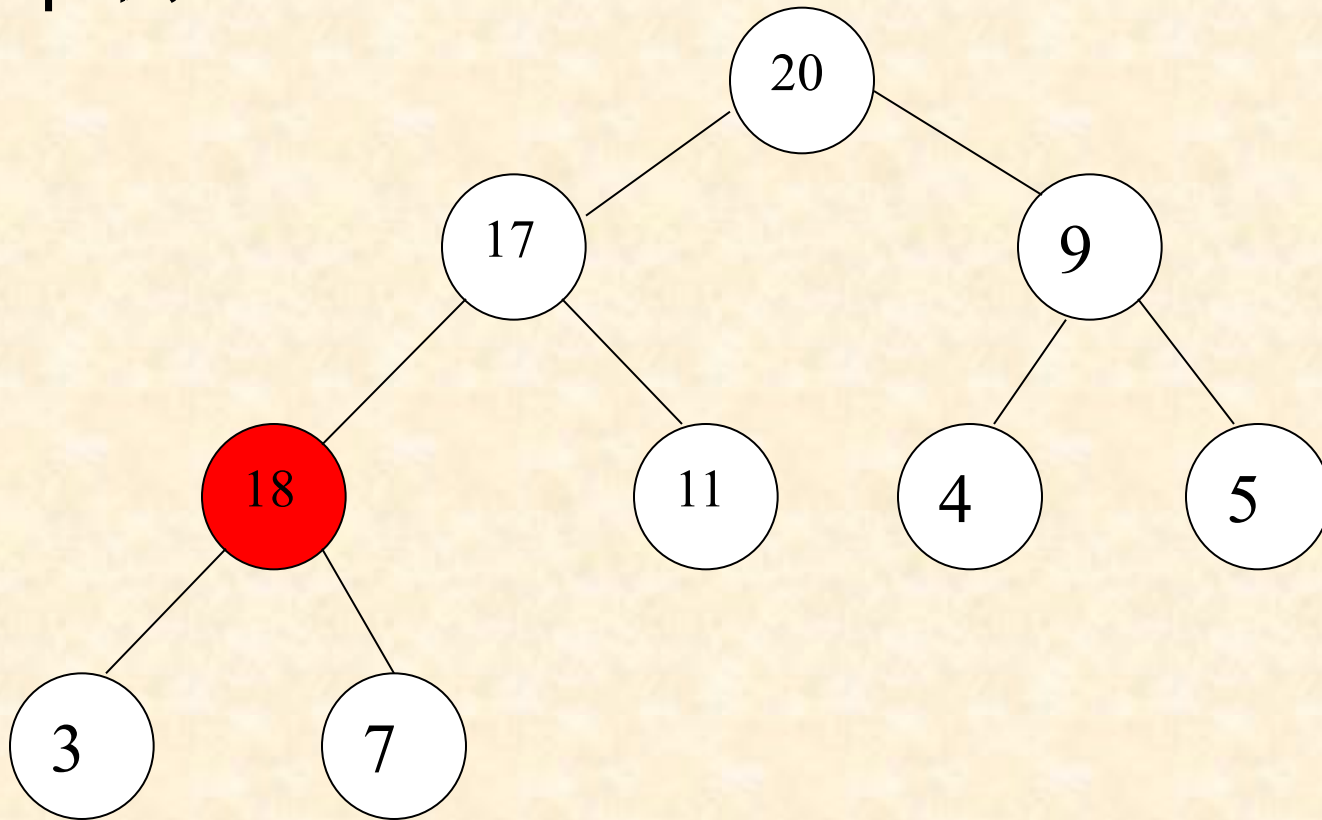


Sift-up 例1



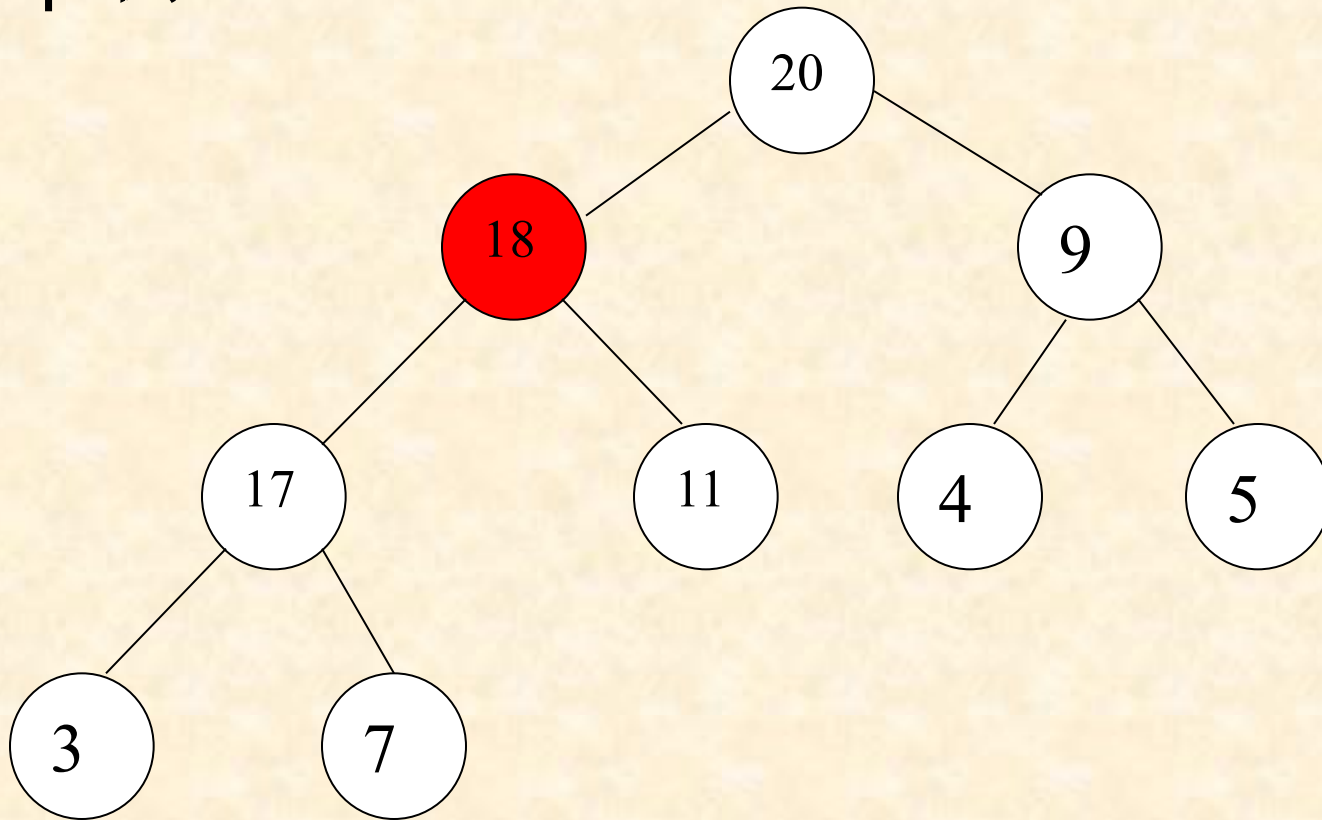


Sift-up 例2





Sift-up 例2





过程 Sift-up(H,i)

输入：数组H[1...n]，索引 $1 \leq i \leq n$

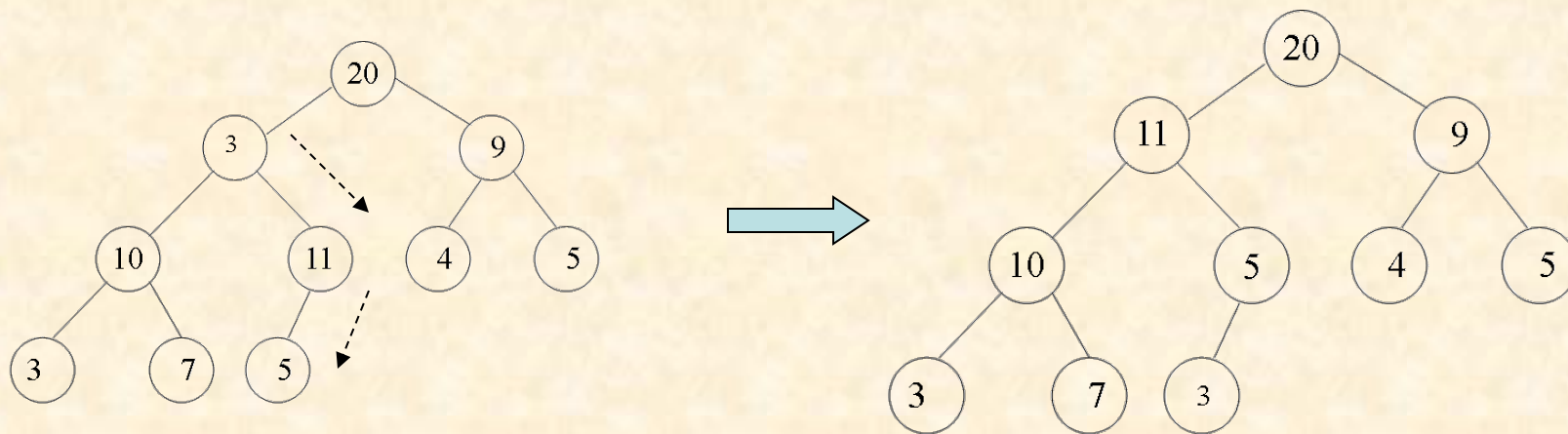
输出：上移H[i] (如果需要)，使它的键值不大于父节点的键值

1. **done** ← false
2. if $i=1$ then exit //根节点
3. **repeat**
4. if $\text{key}(H[i]) > \text{key}(H[\lfloor i/2 \rfloor])$ then 互换 H[i] 和 $H[\lfloor i/2 \rfloor]$
5. else **done** ← true //调整过程至此已经满足要求，可退出
6. $i \leftarrow \lfloor i/2 \rfloor$
7. **until** $i=1$ or **done** //调整进行到根节点，或到某一节点终止



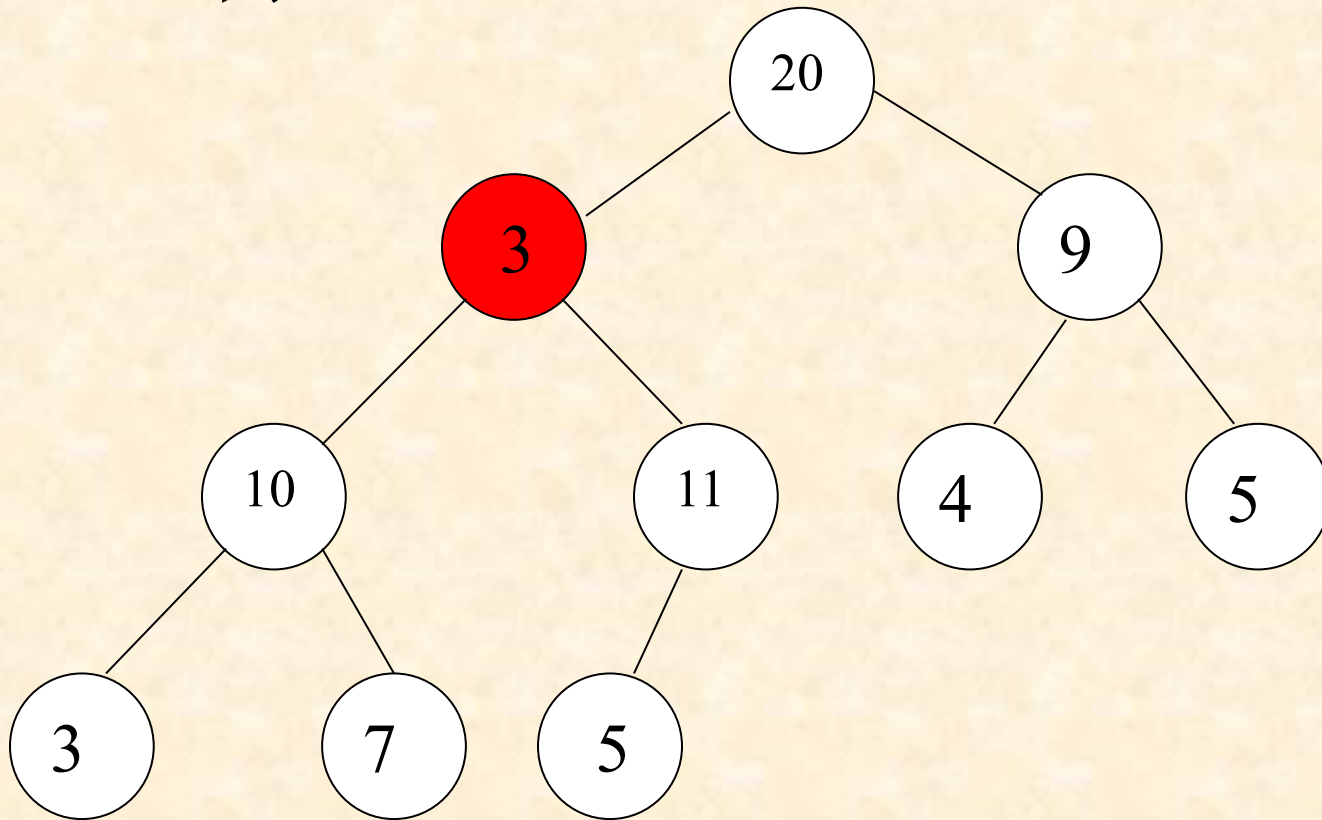
辅助运算Sift-down

- 假如某个内部节点 $H[i]$ ($i \leq \lfloor n/2 \rfloor$), 其键值小于儿子节点的键值, 即 $\text{key}(H[i]) < \text{key}(H[2i])$ 或 $\text{key}(H[i]) < \text{key}(H[2i+1])$ (如果右儿子存在), 违背了堆特性, 需要进行调整。
- 调整方法: 下渗。
- 沿着从 $H[i]$ 到子节点(可能不唯一, 则取其键值较大者)的路径, 比较 $H[i]$ 与子节点的键值, 若 $\text{key}(H[i]) < \max(\text{key}(H[2i]), \text{key}(H[2i+1]))$ 则交换之。这一过程直到叶子节点或满足堆特性为止。



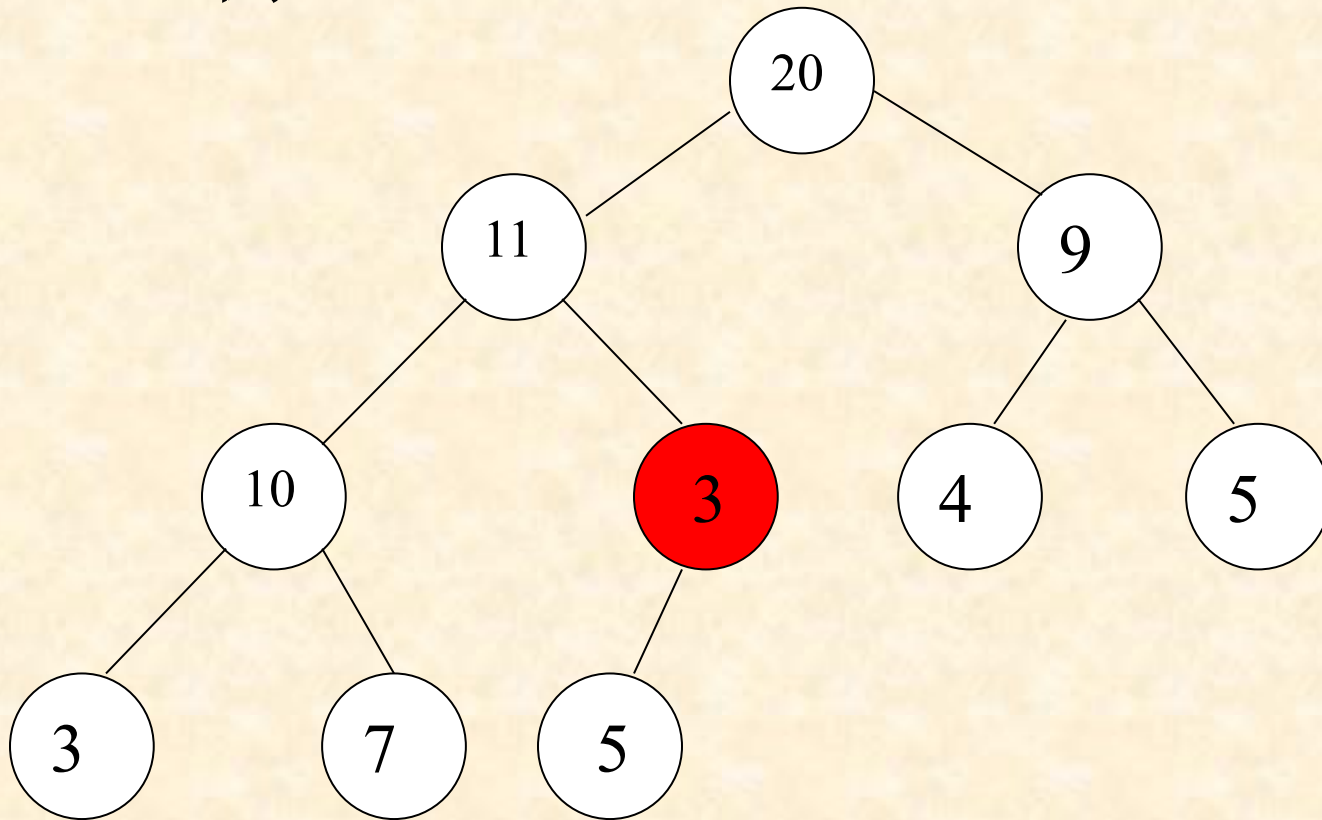


Sift-down 例1



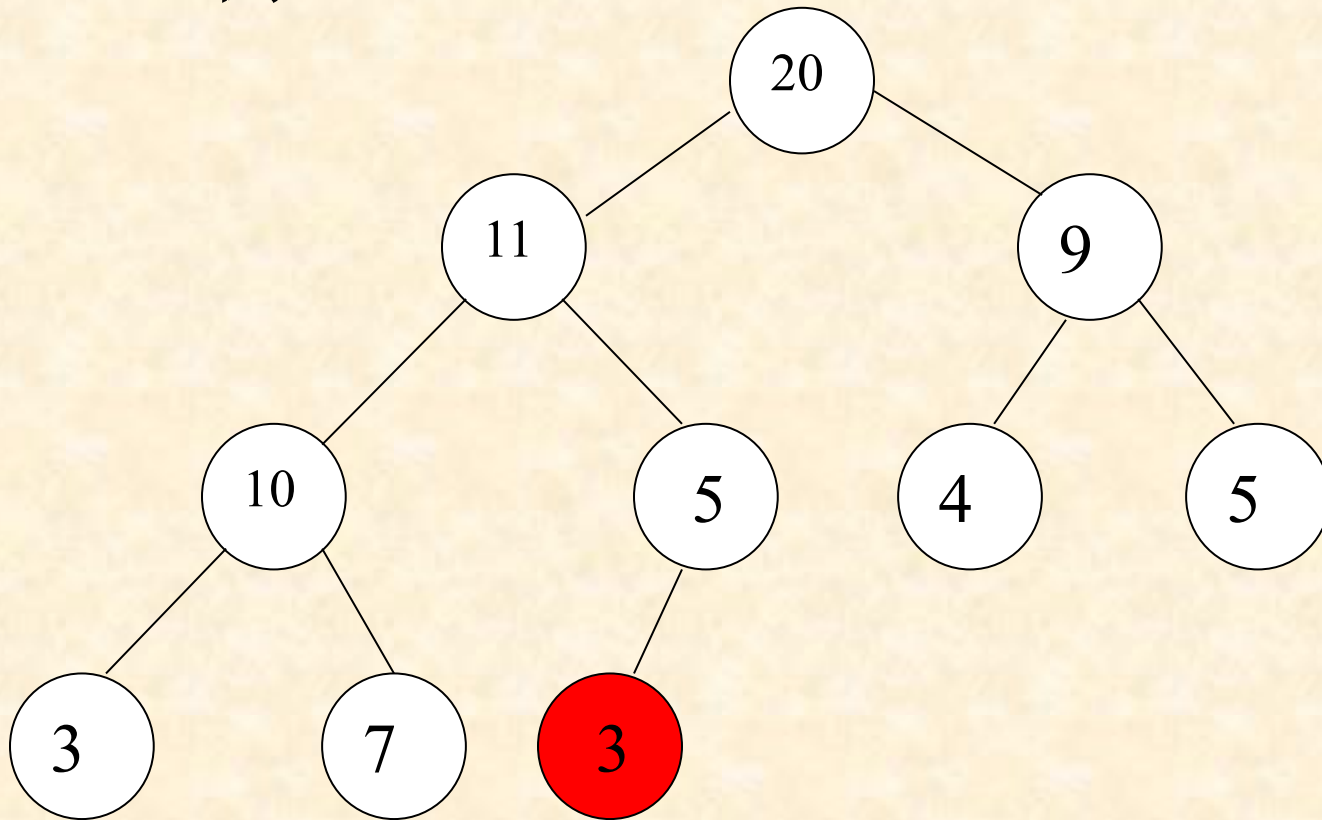


Sift-down 例1



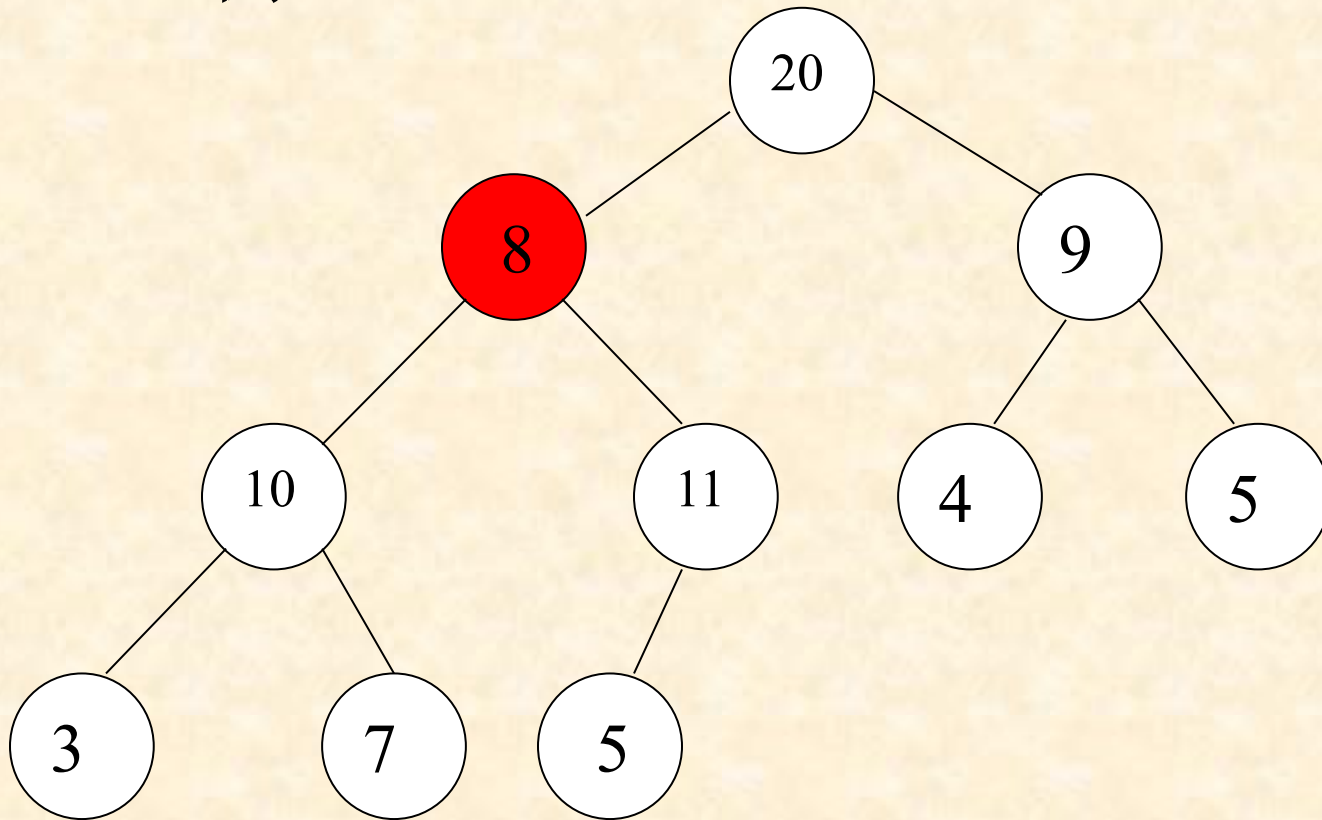


Sift-down 例1



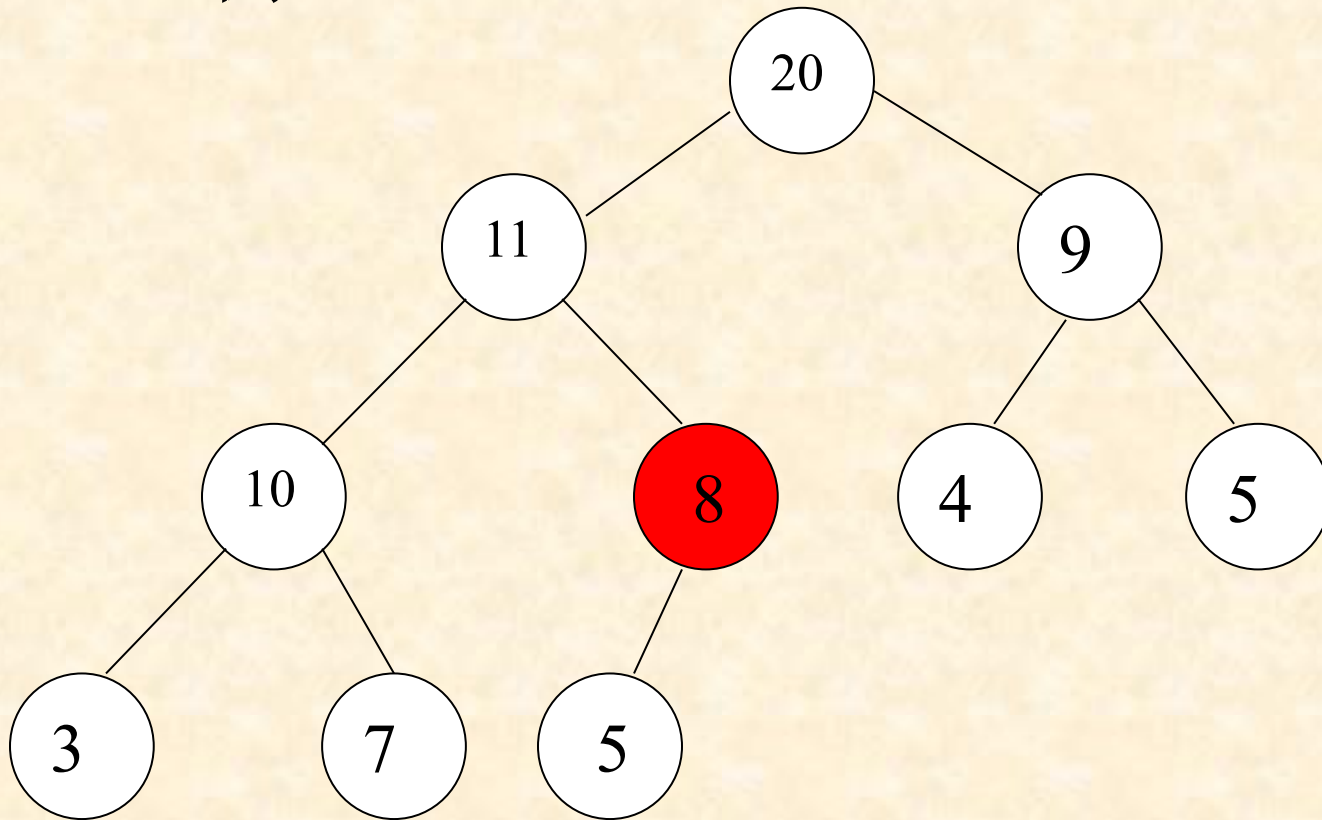


Sift-down 例2





Sift-down 例2





过程 Sift-down(H,i)

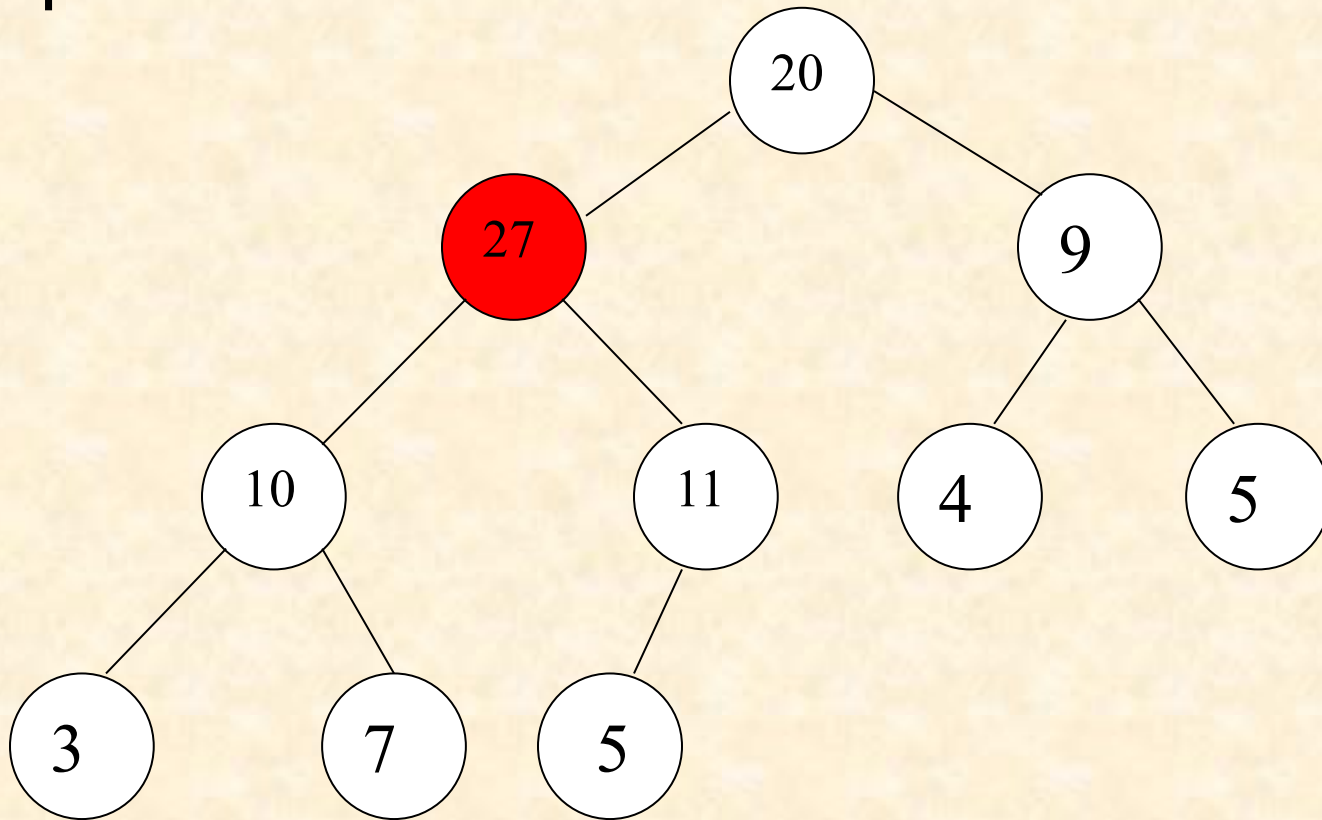
输入：数组H[1...n]，索引 $1 \leq i \leq n$

输出：下渗H[i] (若它违背了堆特性)，使H满足堆特性

1. done $\leftarrow$ false
2. if $2i > n$, then exit //叶子节点，无须进行
3. **repeat**
4. $i \leftarrow 2i$
5. if $i+1 \leq n$ and $\text{key}(H(i+1)) > \text{key}(H(i))$ then $i = i+1$ //有右儿子
6. if $\text{key}(H[\lfloor i/2 \rfloor]) < \text{key}(H[i])$ then 互换 H[i] 和 H[$\lfloor i/2 \rfloor$]
7. else done $\leftarrow$ true //调整过程至此已经满足堆特性，可退出
8. end if
9. **until** $2i > n$ or **done** //调整进行到叶节点，或到某一节点终止

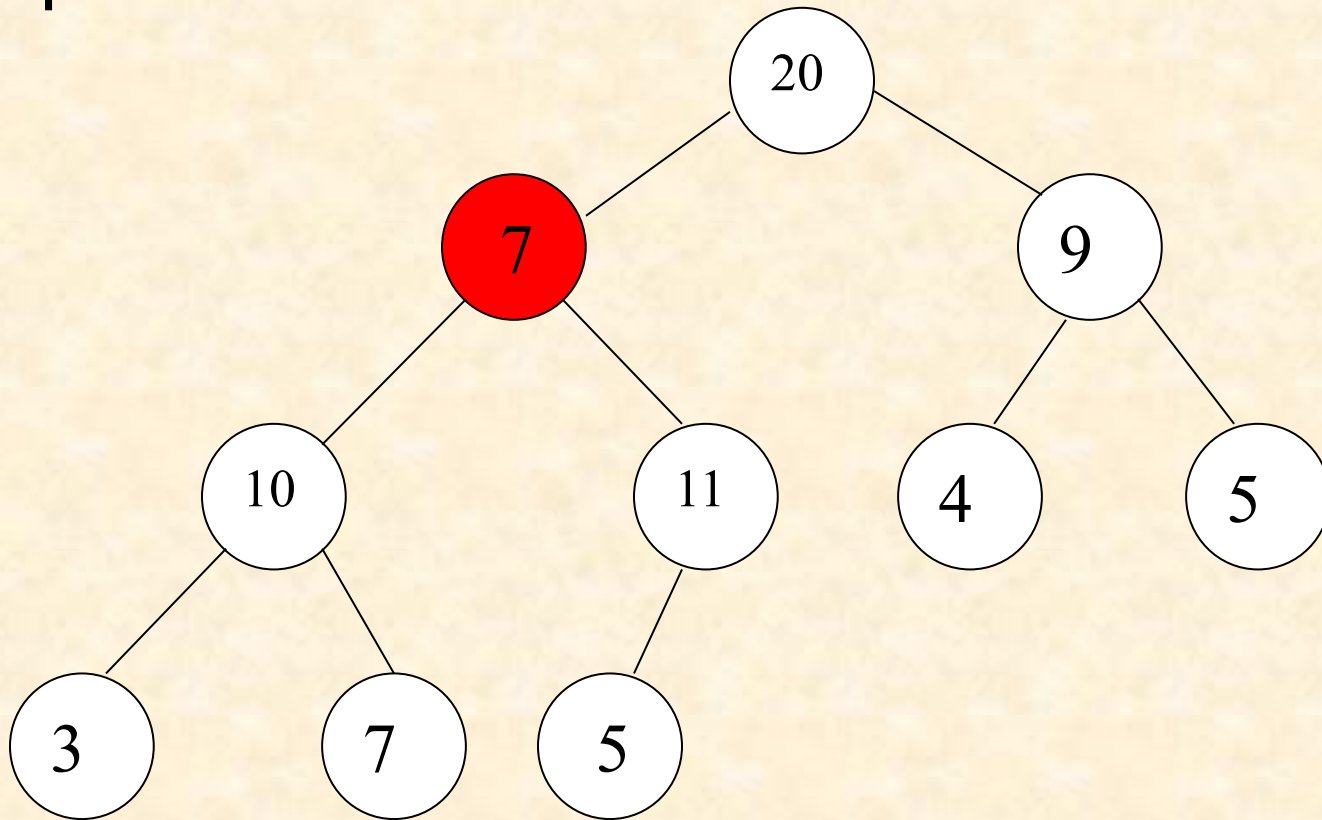


Sift-up or Sift-down ?





Sift-up or Sift-down ?





操作insert(H,x): 插入元素x到堆H中

- 思路：先将x添加到H的末尾，然后利用Sift-up，调整x在H中的位置，直到满足堆特性。

输入：堆 $H[1\dots n]$ 和元素x

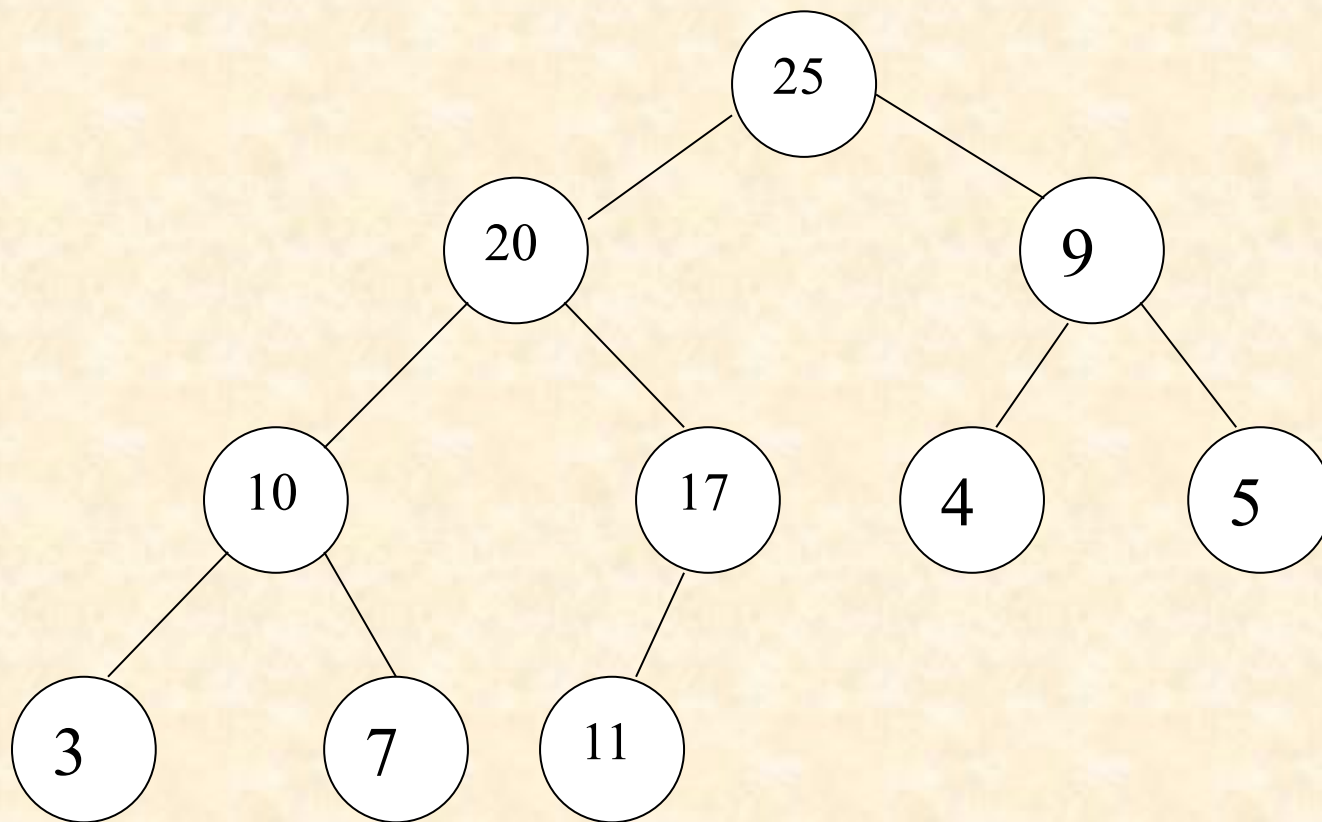
输出：新堆 $H[1\dots n+1]$ ，x是其中元素之一。

1. $n \leftarrow n+1$ {堆大小增1}
2. $H[n] \leftarrow x$;
3. Sift-up(H,n) {调整堆}

树的高度为 $\lfloor \log n \rfloor$, 所以将一个元素插入大小为n的堆所需要的时间是 $O(\log n)$.



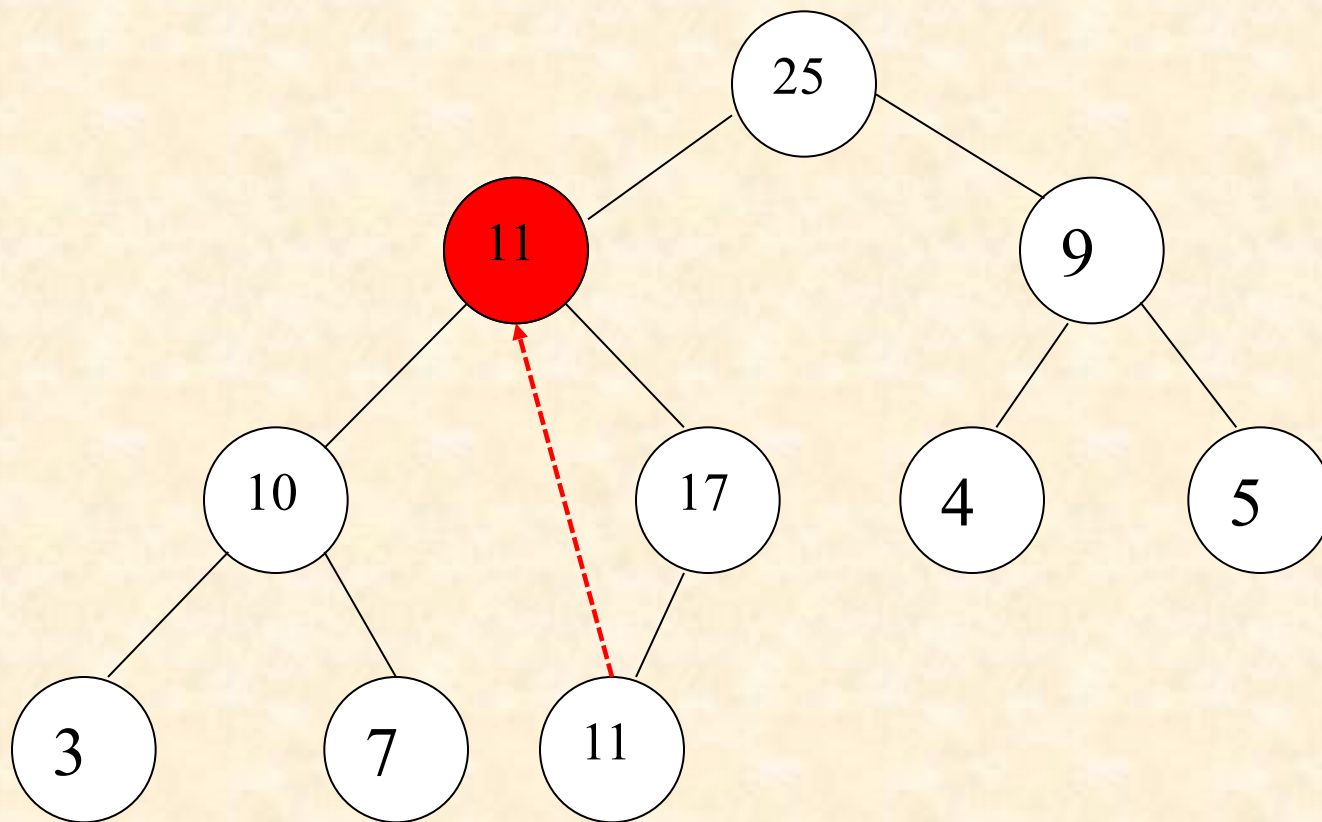
堆的删除操作



观察：删除哪个元素最简单？



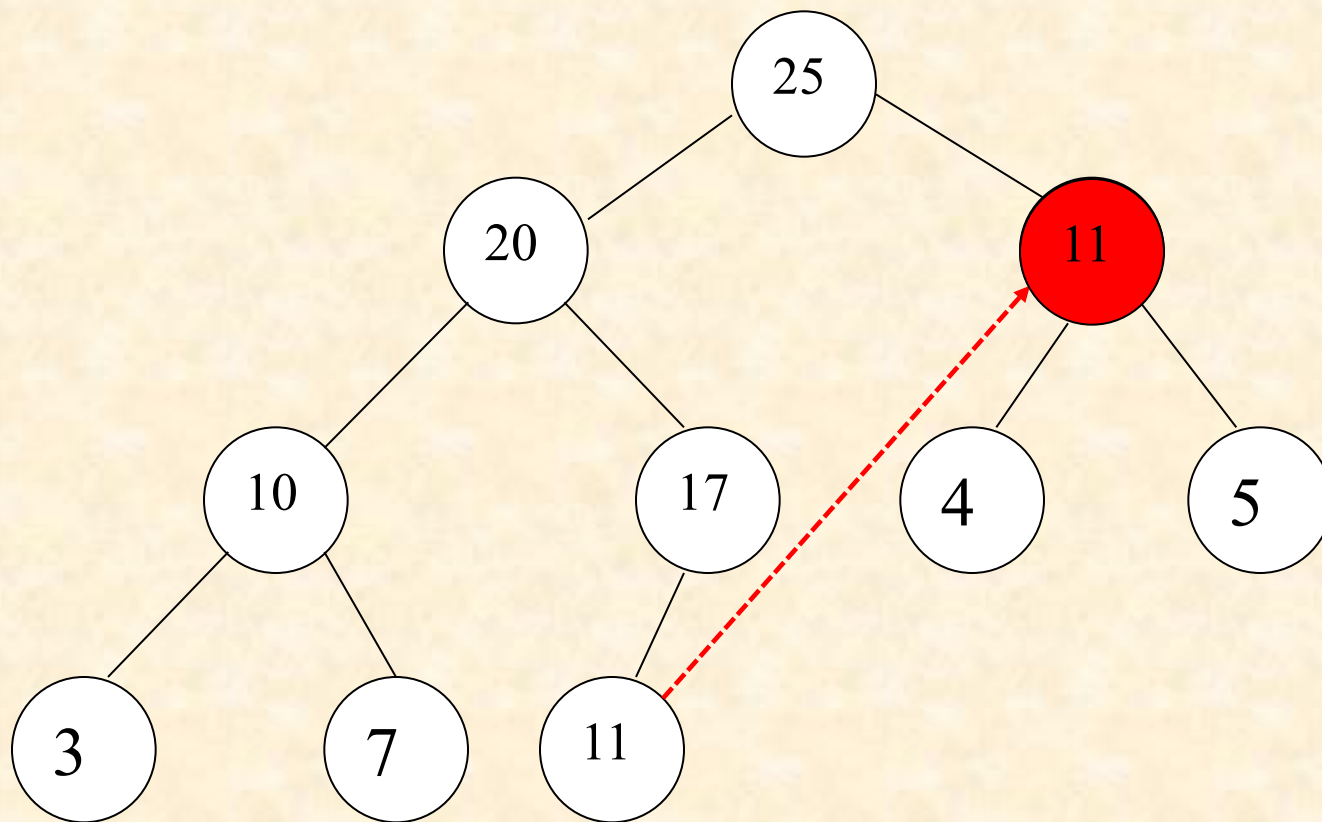
堆的删除操作



删除20，变为删除最后一个元素：11



堆的删除操作



删除9，变为删除最后一个元素：11



操作 delete(H,i)

- 思路：先用 $H[n]$ 取代 $H[i]$ ，然后对 $H[i]$ 作Sift-up或Sift-down)，直到满足堆特性。

输入：非空堆 $H[1\dots n]$ ，索引 i ， $1 \leq i \leq n$ 。

输出：删除 $H[i]$ 之后的新堆 $H[1\dots n-1]$ 。

1. $x \leftarrow H[i]$; $y \leftarrow H[n]$;
2. $n \leftarrow n-1$; //堆大小减1
3. if $i=n+1$ then exit //要删除的刚好是最后一个元素，叶节点
4. $H[i] \leftarrow y$; //用原来的 $H[n]$ 取代 $H[i]$
5. if **key(y) $\geq$ key(x)** then Sift-up(H,i)
6. **else** Sift-down(H,i);
7. end if

所需要的时间是 $O(\log n)$ 。



操作delete-max(H)

输入：堆 $H[1\dots n]$

输出：返回最大键值元素，并将其从堆中删除

1. $x \leftarrow H[1]$

2. delete(H,1)

3. return x



make-heap(A): 从数组A创建堆

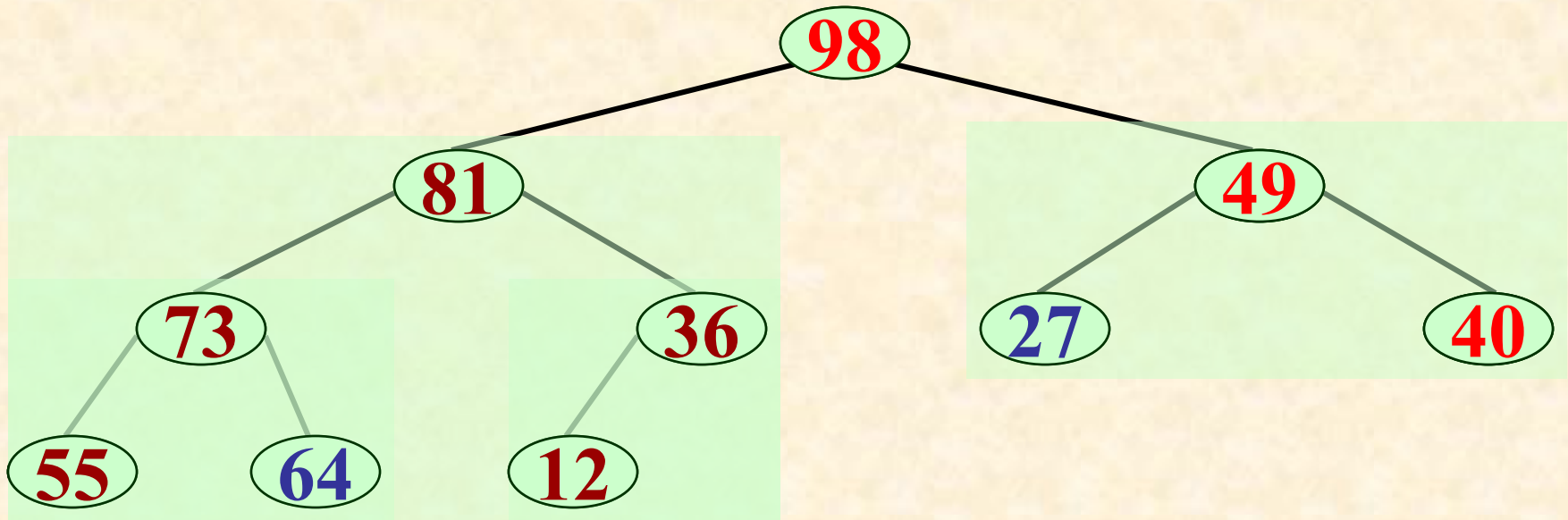
- 方法1: 从一个空堆开始, 逐步插入A中的每个元素, 直到A中所有元素都被转移到堆中。
- 时间复杂度为 $\Theta(n \log n)$. 为什么?

$$\begin{aligned} \sum_{i=2}^n [\log(i-1) - 1] &< \sum_{i=2}^n \lfloor \log(i-1) \rfloor \leq \sum_{i=2}^n \log(i-1) \\ &= \sum_{i=1}^{n-1} \log i = \Theta(n \log n) \end{aligned}$$



建堆是一个从下往上进行“筛选”的过程。

例如：排序之前的关键字序列为



现在，左/右子树都已经调整为堆，最后只要调整根结点，使整个二叉树是个“堆”即可。



方法2:

MAKEHEAP (创建堆)

输入: 数组 $A[1\dots n]$

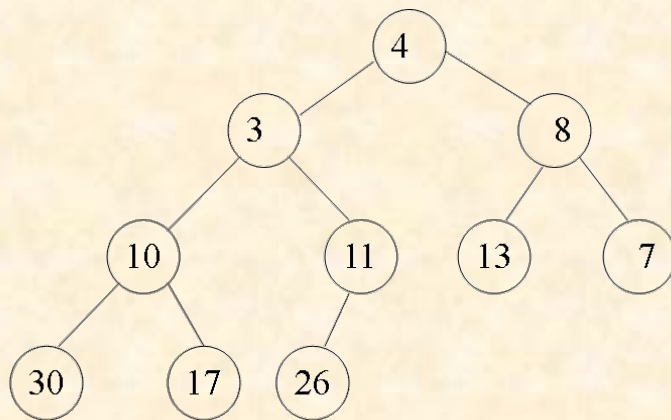
输出: 将 $A[1\dots n]$ 转换成堆

1. for $i \leftarrow \lfloor n/2 \rfloor$ downto 1

2. Sift-down(A, i) {使以 $A[i]$ 为根节点的子树调整成为堆, 故调用down过程}

3. end for

例: 给定数组 $A[1\dots 10] = \{4, 3, 8, 10, 11, 13, 7, 30, 17, 26\}$





复杂度分析

- 树高 $k = \lfloor \log n \rfloor$, 第 i 层正好 2^i 个节点, $0 \leq i < k$, (不含最深的叶子节点层), 每个节点的下沉动作最多执行 $k-i$ 次, 故下沉动作执行次数上限为

$$\sum_{j=0}^n c^j = \frac{c^{n+1} - 1}{c - 1} = \Theta(c^n), c \neq 1$$

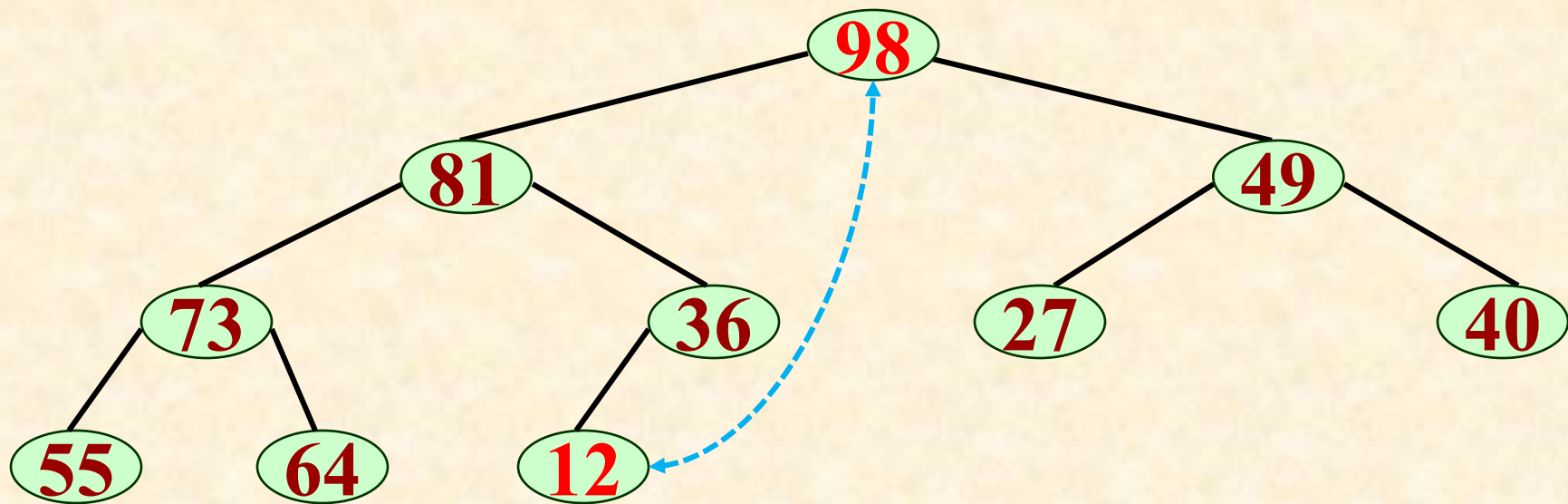
$$\Rightarrow c \cdot \frac{d}{dc} \sum_{j=0}^n c^j = \sum_{j=0}^n j c^j$$

$$= \frac{[n c^{n+2} - n c^{n+1} - c^{n+1} + c]}{(c-1)^2}$$

- 令 $c = 1/2 \Rightarrow \sum_{j=0}^n j 2^{-j} = 2 - (n+2)/2^n$

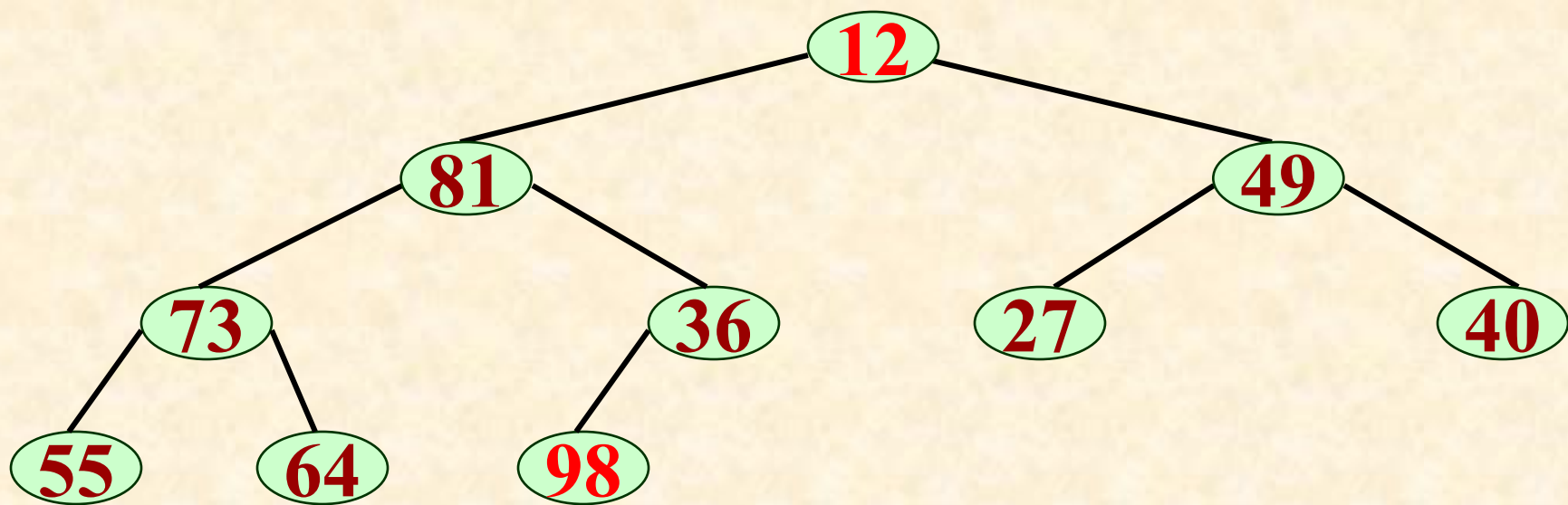


堆排序是逐步把堆的最大值置于末尾的过程。



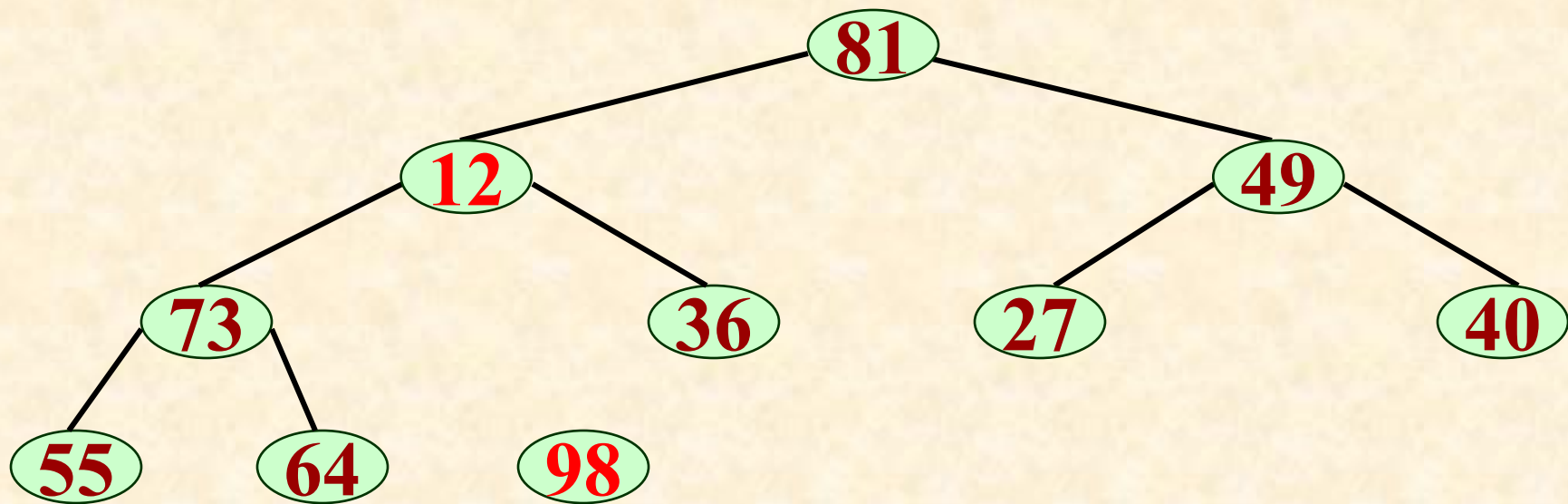


堆排序是逐步把堆的最大值置于末尾的过程。



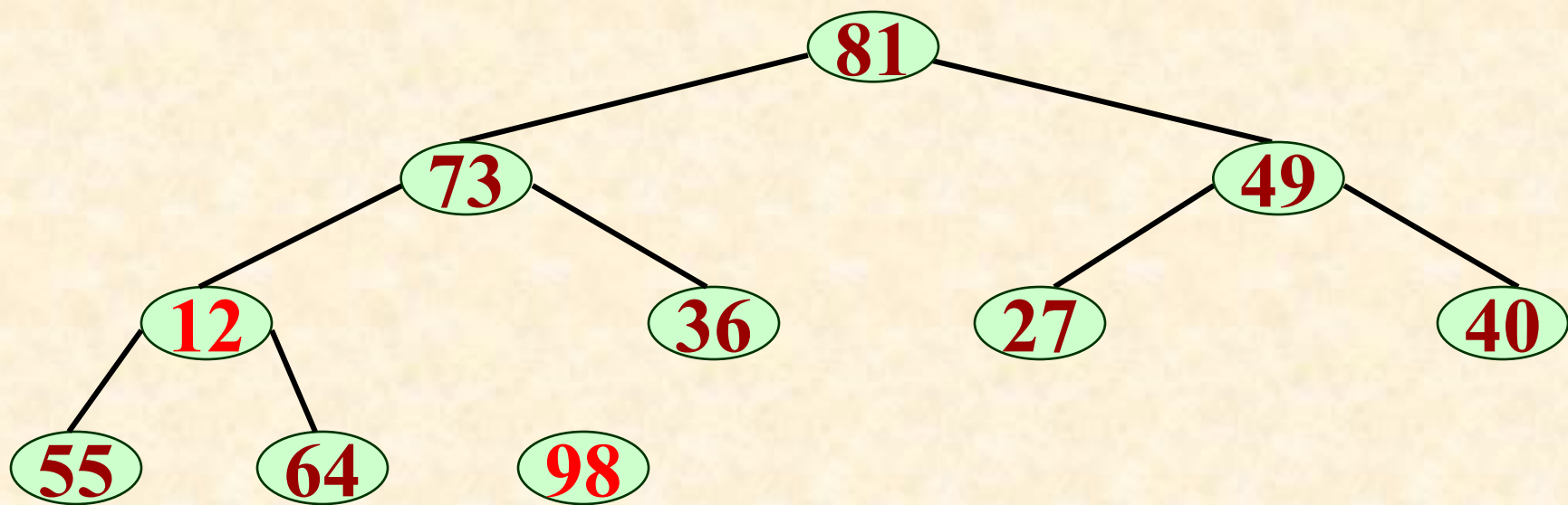


堆排序是逐步把堆的最大值置于末尾的过程。



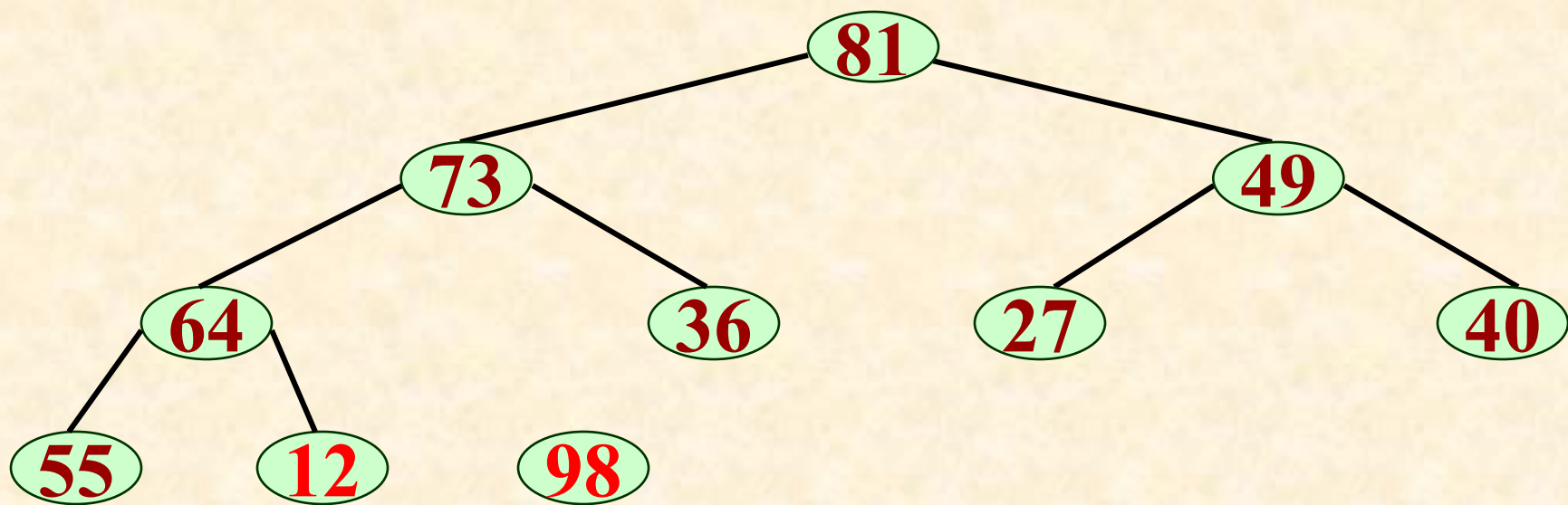


堆排序是逐步把堆的最大值置于末尾的过程。



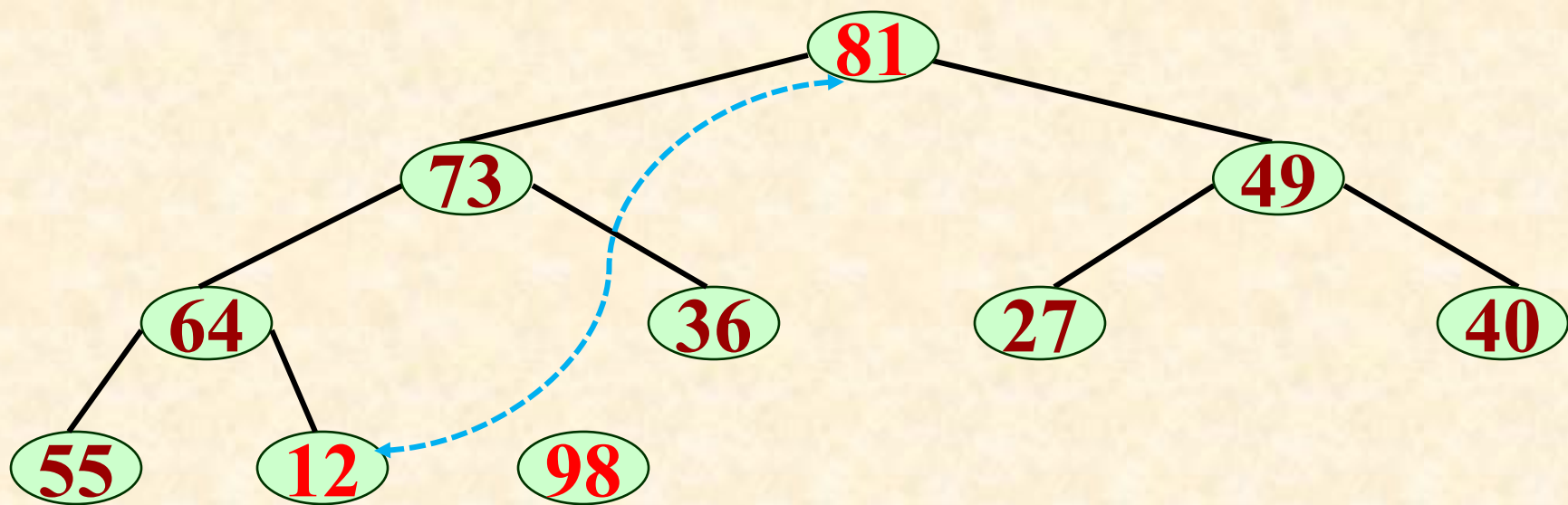


堆排序是逐步把堆的最大值置于末尾的过程。



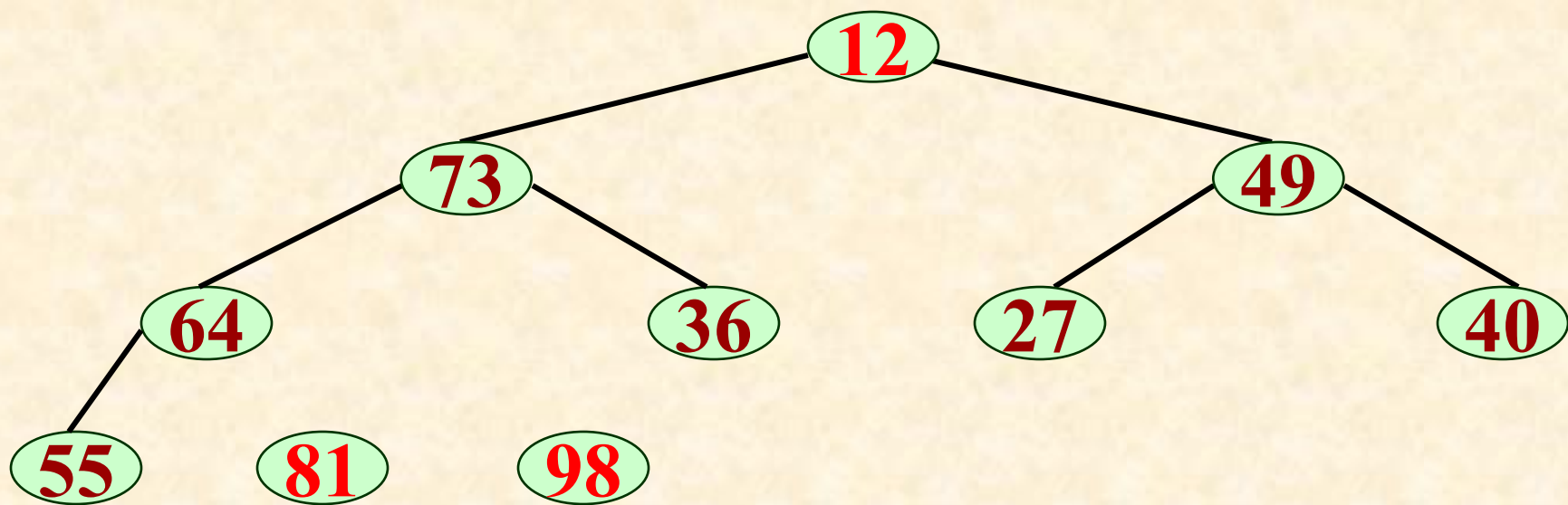


堆排序是逐步把堆的最大值置于末尾的过程。



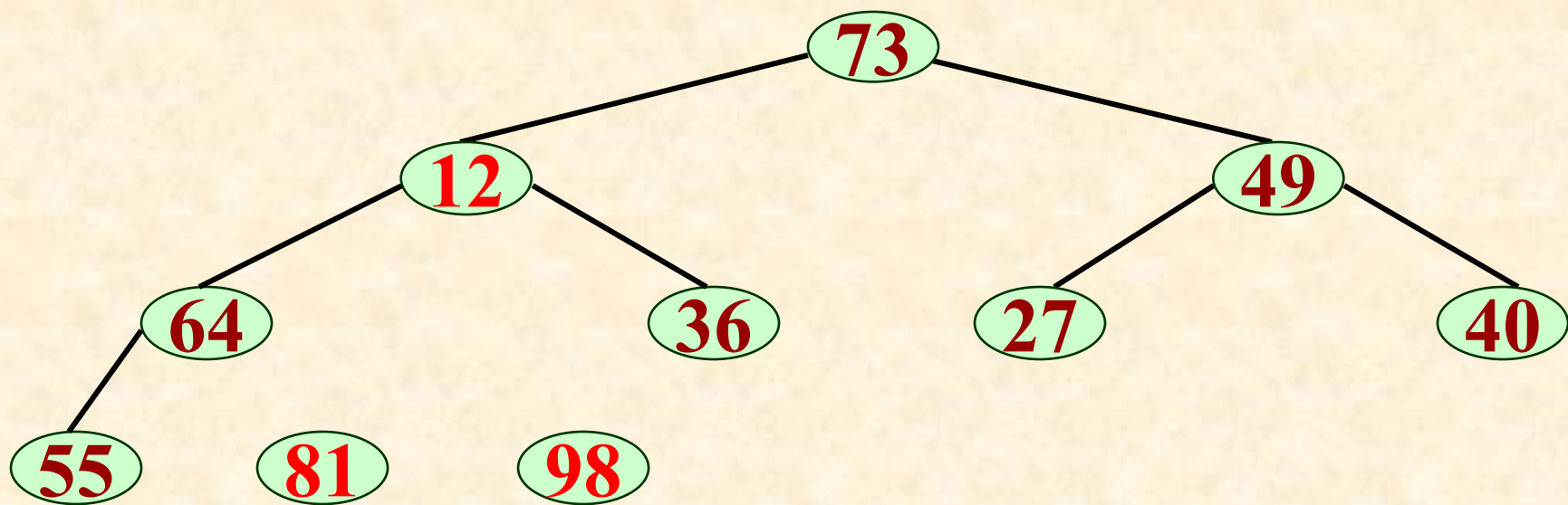


堆排序是逐步把堆的最大值置于末尾的过程。



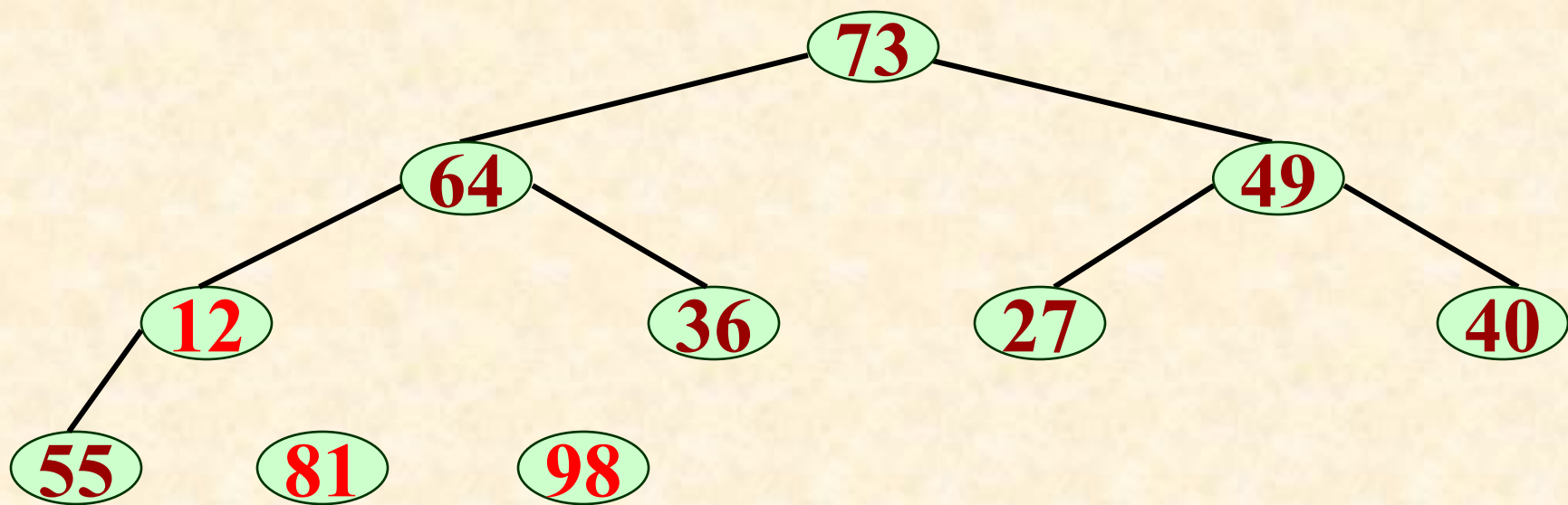


堆排序是逐步把堆的最大值置于末尾的过程。



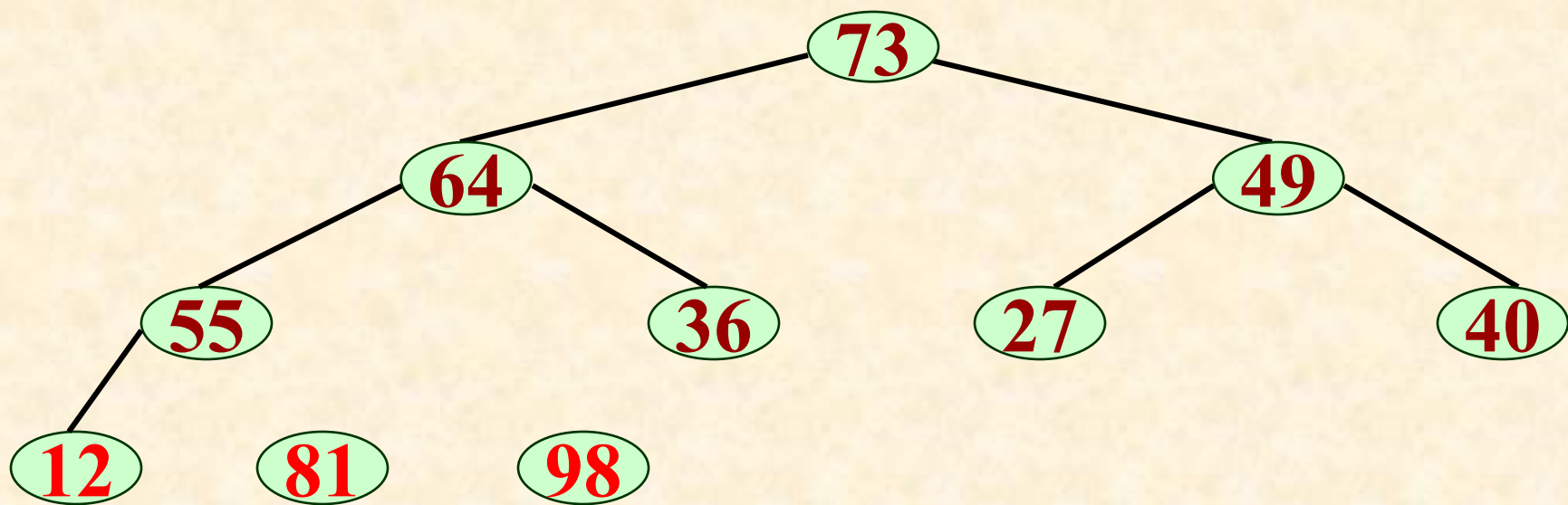


堆排序是逐步把堆的最大值置于末尾的过程。



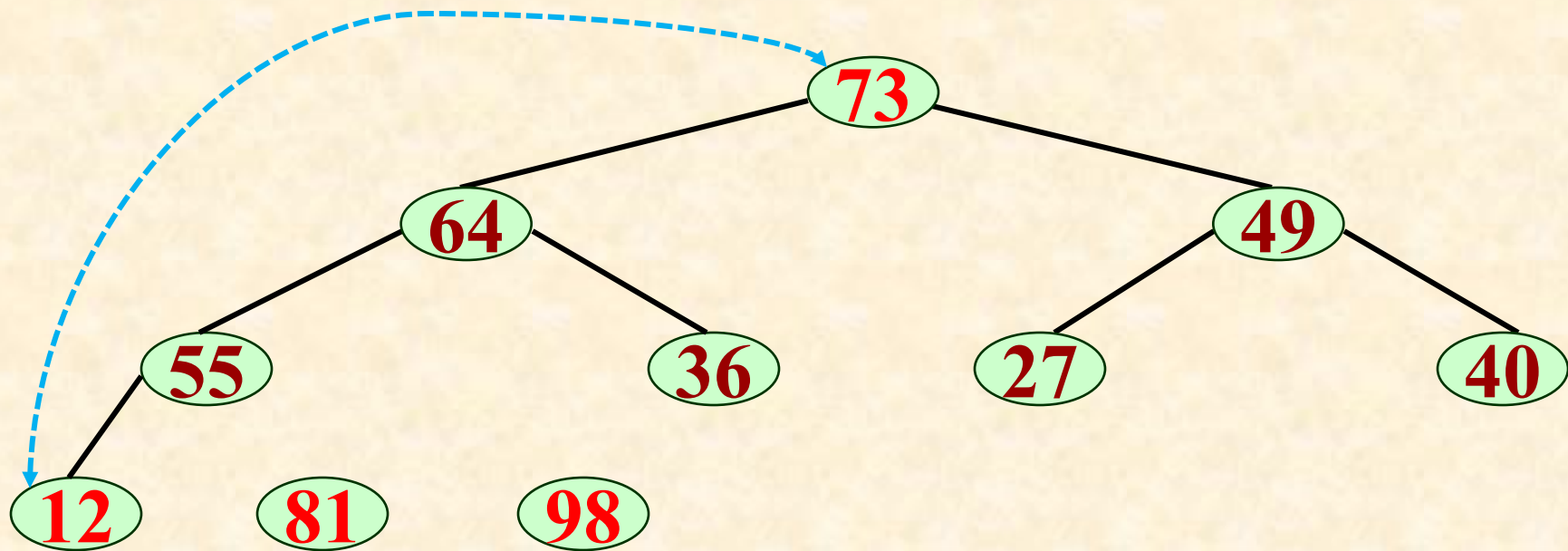


堆排序是逐步把堆的最大值置于末尾的过程。



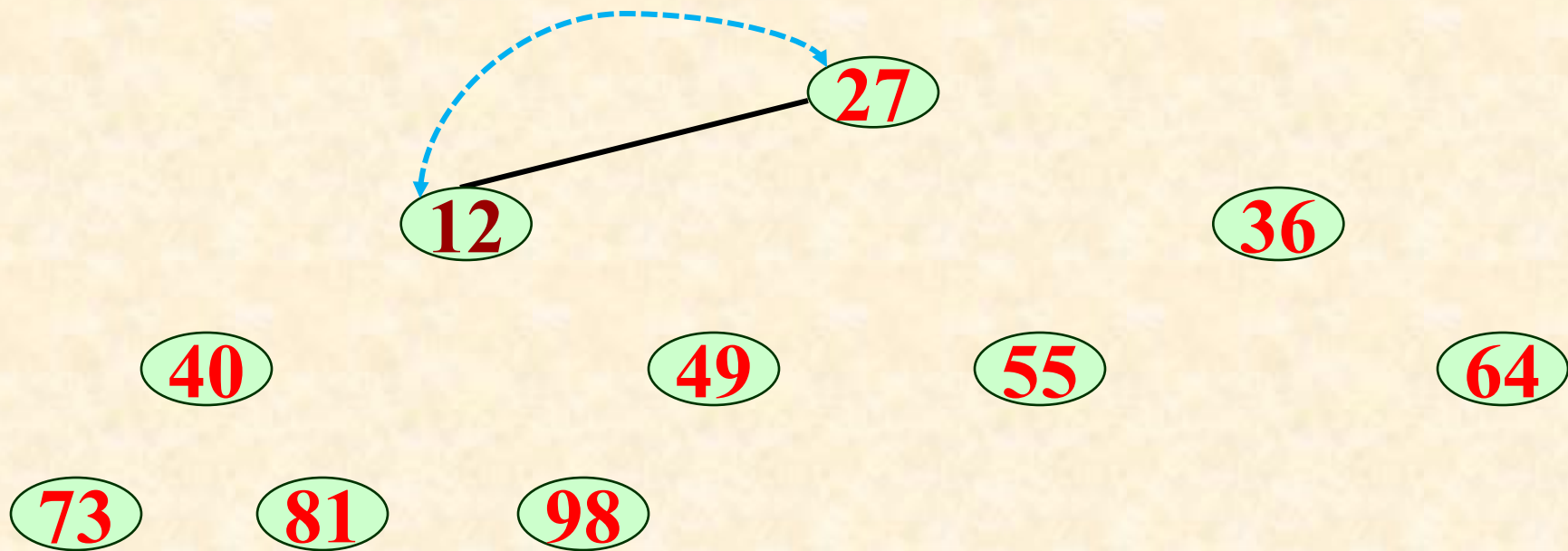


堆排序是逐步把堆的最大值置于末尾的过程。





直到某一时刻，世界变成这个样子...





然后...





堆排序

输入：数组 $A[1\dots n]$

输出：非降序排列的数组

1. MAKEHEAP(A)

2. for $j \leftarrow n$ downto 2

3. 互换 $A[1]$ 和 $A[j]$

4. Sift-down($A[1\dots j-1], 1$) {下渗根节点 $A[1]$ }

5. end for

时间复杂度: $\Theta(n) + (n-1)O(\log n) = O(n \log n)$

空间复杂度: $\Theta(1)$ (原地排序, 无需额外辅助空间)

$$f(n) = \lfloor \log(n-1) \rfloor + \lfloor \log(n-2) \rfloor + \dots + \lfloor \log 2 \rfloor + \lfloor \log 1 \rfloor$$

$$\log x - 1 < \lfloor \log x \rfloor \leq \log x$$

$$\therefore f(n) \leq \log(n-1) + \log(n-2) + \dots + \log 2 + \log 1$$

$$< \sum_{j=1}^n \log j = \Theta(n \log n)$$

$$\therefore f(n) > \log(n-1) - 1 + \log(n-2) - 1 + \dots + \log 2 - 1 + \log 1 - 1$$

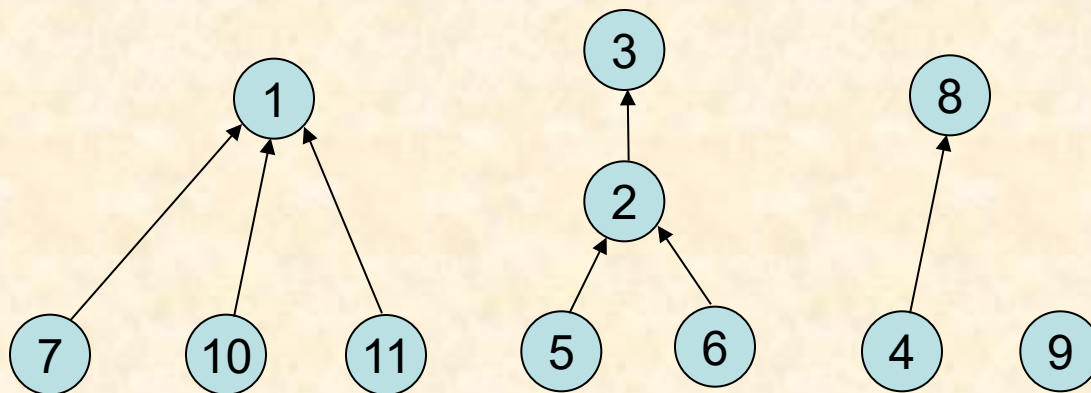
$$= \sum_{j=1}^{n-1} \log j - (n-1) = \Theta(n \log n)$$

$$\therefore f(n) = \Theta(n \log n)$$



不相交集(Disjoint Sets)

- 假设有 n 个元素，被分成若干个集合。例如 $S=\{1,2,\dots,11\}$ 分成4个子集 $1:\{1,7,10,11\}$ ， $3:\{2,3,5,6\}$ ， $8:\{4,8\}$ ， $9:\{9\}$ 并分别命名。
- 事实上，每个子集可以用树表示，除根节点外，每个节点都有指针指向父节点。上例可以用树表示为：

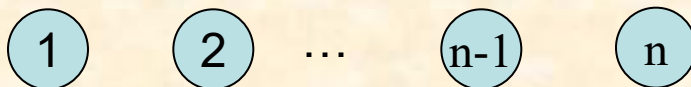




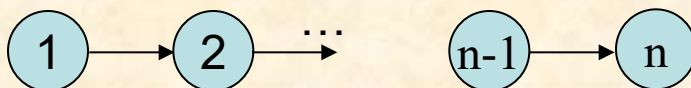
- 假如要执行如下计算任务：
 - $\text{FIND}(x)$: 寻找包含元素 x 的集合的名字
 - $\text{UNION}(x,y)$: 将包含元素 x 和 y 的两个集合合并，重命名。
- 记 $\text{root}(x)$ 为包含元素 x 的树的根，则 $\text{FIND}(x)$ 返回 $\text{root}(x)$.
- 执行合并 $\text{UNION}(x, y)$ 时，首先依据 x 找到 $\text{root}(x)$,记为 u ,依据 y 找到 $\text{root}(y)$ ，记为 v ；然后，将 u 指向 v 。
- 优点：简单明了
- 缺点：多次合并后，树高度可能很大，查找困难。



例：初始状态： $\{1\}, \{2\}, \dots, \{n\}$



执行合并序列： $\text{UNION}(1,2), \text{UNION}(2,3), \dots, \text{UNION}(n-1,n)$. 我们得到的结果是：



执行查找序列： $\text{FIND}(1)$ // 返回 $\text{root}(1)$!, $\text{FIND}(2), \dots, \text{FIND}(N)$. 需要比较的次数是：

$$n + (n-1) + \dots + 2 + 1 = \frac{n(n+1)}{2}$$

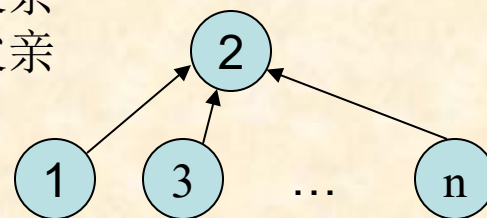
目标：降低树的高度。措施：Rank Heuristic。

1. 给每个树的根节点定义一个秩(rank)，表示该树的高度。
2. 在执行 $\text{UNION}(x, y)$ ，首先找到 $u = \text{root}(x)$ ， $v = \text{root}(y)$ 。
3. 然后比较 $\text{rank}(u)$ 和 $\text{rank}(v)$

若 $\text{rank}(u) = \text{rank}(v)$ ，则使 u 指向 v ， v 成为 u 的父亲，同时 $\text{rank}(v) + 1$

若 $\text{rank}(u) < \text{rank}(v)$ ，则使 u 指向 v ， v 成为 u 的父亲

若 $\text{rank}(u) > \text{rank}(v)$ ，则使 v 指向 u ， u 成为 v 的父亲





Algorithm: UNION

输入：两个元素 x, y .

输出：将包含 x, y 的两棵树合并

1. $u \leftarrow \text{FIND}(x); v \leftarrow \text{FIND}(y)$
2. if $\text{rank}(u) \leq \text{rank}(v)$ then // Rank Heuristic
3. $p(u) \leftarrow v$
4. if $\text{rank}(u) = \text{rank}(v)$ then $\text{rank}(v) = \text{rank}(v) + 1$
5. else $p(v) \leftarrow u$
6. end if

目标：进一步提高FIND的操作的性能。措施：在执行FIND操作时，同时进行路径压缩(Path compression)。





Algorithm: FIND

输入：节点 x

输出：root(x)和路径压缩后的树

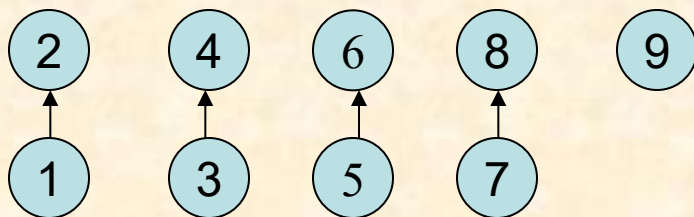
1. $y \leftarrow x$
2. while $p(y) \neq \text{null}$ {寻找包含 x 节点的树的根}
3. $y \leftarrow p(y)$
4. end while
5. $\text{root} \leftarrow y$; $y \leftarrow x$ {重新赋值为原来的节点 x , y 是工作节点}
6. while $p(y) \neq \text{null}$ {执行路径压缩}
7. $w \leftarrow p(y)$ {父节点暂存为 w } //板书图示
8. $p(y) \leftarrow \text{root}$ {令 $p(y)$ 即 y 的父亲等于根节点, 即直接指向根节点}
9. $y \leftarrow w$ {继续下一步压缩}
10. end while
11. return root



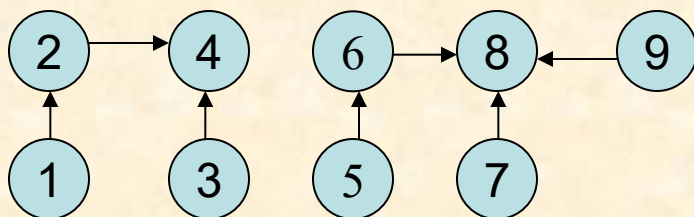
例：初始状态：{1},{2},...,{9}



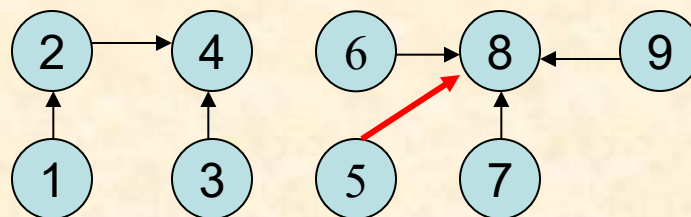
执行合并序列：UNION(1,2),UNION(3,4),UNION(5,6),UNION(7,8),得到的结果是：



继续执行合并序列：UNION(2,4),UNION(8,9),UNION(6,8),得到的结果是：



继续执行：FIND(5)得到的结果是：

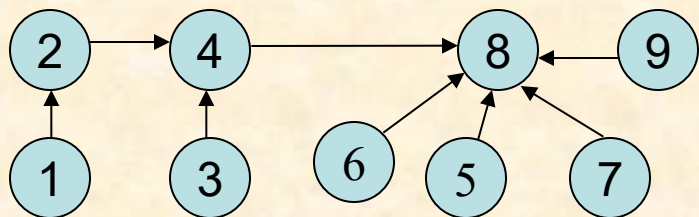


注意：路径压缩时，秩不会改变。

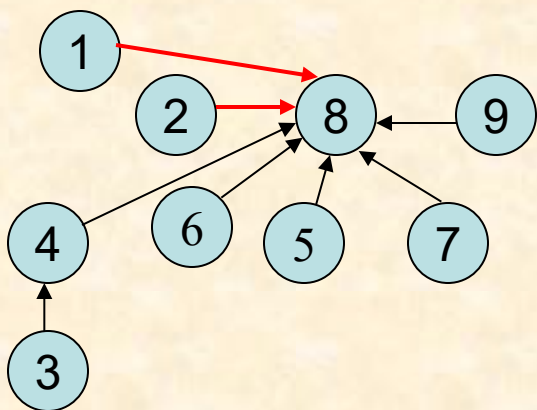
即执行FIND操作后，根节点的秩有可能大于树的高度。



继续执行：UNION(4,8)得到的结果是：



继续执行：FIND(1)得到的结果是：





作业

- **4.7, 4.11, (4.13)**